**UNIVERSITÀ POLITECNICA DELLE MARCHE**

**DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE**

Corso di Laurea Magistrale in Ingegneria Informatica e dell'Automazione

**Insegnamento di New Generation Databases**




**Presentato da:**

**Andrea Altieri**

**Andrea Flaiani**

**Docente:**

**Prof.ssa Claudia Diamantini**

# ANOMALY DETECTION SU DATI BIOMETRICI

## RELAZIONE DI PROGETTO

**Anno Accademico 2026/2027**

# Indice


**1 Introduzione e dominio applicativo** **1**

1.1 Contesto e motivazioni del progetto . . . 1
1.2 Il dominio clinico e il dataset VitalDB . . . 1
1.3 Obiettivi e struttura dell'elaborato . . . 3

**2 Architettura della data pipeline** **4**

2.1 Il paradigma della medallion architecture . . . 4
2.2 Layer Bronze per l'ingestione e la storicizzazione dei dati grezzi . . . 5
2.3 Layer Silver per la bonifica e il partizionamento dei dati . . . 6
2.4 Evoluzione del modello dati da tabella piatta a documento BSON . . . 8

**3 Archiviazione e governance con MongoDB Time Series** **10**

3.1 Layer Gold e archiviazione con MongoDB Time Series . . . 10
3.2 Benchmark prestazionale e di occupazione dello storage . . . 11
3.3 Data governance artigianale in ottica data lakehouse . . . 12
3.3.1 Catalogo e registrazione della provenienza (collezione registry) . . . 13
3.3.2 Metadati descrittivi delle tracce (collezione track_metadata) . . . 14
3.3.3 Limiti transazionali e strategia applicativa di compensazione . . . 14
3.4 Pipeline di aggregazione analitica e ottimizzazione delle query . . . 14
3.4.1 Downsampling temporale efficiente lato database con $dateTrunc . . . 14
3.4.2 Ottimizzazione dell'accesso anagrafico ed eliminazione del problema N+1 . . . 15
3.5 Uso dei metadati per la data quality e analisi critica . . . 16

**4 Modelli di anomaly detection** **18**

4.1 Approccio ibrido e multilivello al rilevamento di anomalie . . . 18
4.2 Regole cliniche dominio-specifiche . . . 18
4.2.1 Shock Index clinico . . . 18
4.2.2 Ipotensione severa . . . 19
4.3 Ingegnerizzazione delle feature temporali . . . 19
4.4 Modelli di machine learning non supervisionato . . . 20
4.4.1 Isolation Forest . . . 20
4.4.2 LSTM Autoencoder neurale (PyTorch) . . . 20
4.5 Valutazione empirica, metriche e concordanza . . . 22
4.6 Persistenza e tracciamento delle anomalie su MongoDB . . . 24

**5 Ingegnerizzazione del sistema, interfaccia grafica e conclusioni** **26**

5.1 Backend REST API con FastAPI . . . 26
5.2 Dashboard web e interfaccia utente interattiva . . . 27
5.2.1 Vista di telemetria clinica e anomaly detection . . . 29

5.2.2 Vista di data lakehouse, governance e benchmark . . . . . 29
5.3 Containerizzazione ed orchestrazione con Docker . . . . . 29
5.4 Conclusioni, limiti e sviluppi futuri . . . . . 31

CAPITOLO 1

# Introduzione e dominio applicativo

## 1.1 Contesto e motivazioni del progetto

Il monitoraggio fisiologico continuo in ambito operatorio e nei reparti di terapia intensiva genera una mole imponente di dati temporali ad alta frequenza. Durante un intervento chirurgico, monitor multiparametrici registrano senza sosta segnali biometrici vitali: battito cardiaco, concentrazione di ossigeno nel sangue e andamento della pressione arteriosa. La disponibilità di queste serie storiche dischiude grandi opportunità per la medicina computazionale, ma al contempo pone sfide ingegneristiche complesse legate alla gestione, alla qualità e alla tempestività dell'analisi del dato.

I dati clinici grezzi raccolti in sala operatoria sono intrinsecamente rumorosi e disomogenei. Il movimento del paziente o dell'operatore, il distacco accidentale degli elettrodi e le interferenze strumentali provocano artefatti evidenti, dispersioni e lunghi intervalli di valori mancanti. A ciò si aggiunge una marcata asimmetria nelle frequenze di campionamento: mentre la frequenza cardiaca e la saturazione periferica dell'ossigeno vengono campionate con cadenza quasi continua ogni uno o due secondi, le misurazioni pressorie non invasive avvengono a intervalli discreti e dilatati, tipicamente ogni due o cinque minuti, a causa dei tempi necessari per il gonfiaggio e lo sgonfiaggio del bracciale pneumatico.

In un simile scenario, le tradizionali architetture di archiviazione faticano a garantire al tempo stesso l'integrità del dato, l'efficienza dello storage e la reattività richiesta da interrogazioni analitiche complesse. Il presente progetto si colloca nell'alveo dei database di nuova generazione, affrontando l'ingegnerizzazione completa di una piattaforma per l'ingestione, la bonifica, la catalogazione e l'analisi automatizzata di serie storiche biometriche. L'obiettivo primario consiste nel superare la frammentazione tipica dei sistemi ospedalieri mediante un flusso strutturato a stadi, fondato sui principi del **data lakehouse**, associando una rigorosa disciplina di **data governance** all'impiego congiunto di regole cliniche ed algoritmi di **machine learning** per l'individuazione precoce delle instabilità emodinamiche.

## 1.2 Il dominio clinico e il dataset VitalDB

Per lo sviluppo e la validazione del sistema è stato selezionato il dataset clinico aperto **VitalDB**, un archivio ad accesso pubblico contenente registrazioni fisiologiche ad alta fedeltà relative a oltre 6.300 interventi chirurgici non cardiaci e cardiaci, raccolte presso il Seoul National University Hospital tra il 2016 e il 2019.

La scelta di VitalDB è maturata a valle di un confronto comparativo con il dataset alternativo MIMIC-IV. Sebbene quest'ultimo rappresenti un punto di riferimento altrettanto autorevole nella letteratura medica, esso impone un percorso articolato di abilitazione attraverso la piattaforma PhysioNet, con corsi accreditati di etica e protezione dei dati che avrebbero dilatato i tempi iniziali del progetto. Al contrario, VitalDB fornisce un accesso immediato e programmatico mediante una libreria ufficiale in linguaggio Python, garantendo al contempo una granularità temporale e una varietà di parametri pienamente equivalenti e ideali per la sperimentazione di architetture dati avanzate.

In pieno accordo con gli obiettivi del corso, l'attenzione è stata deliberatamente circoscritta all'elaborazione dei parametri biometrici numerici già estratti e standardizzati dalla strumentazione di sala, escludendo le forme d'onda grezze ad altissima frequenza come i tracciati elettrocardiografici o fotopletismografici campionati a centinaia di hertz. L'elaborazione di segnali grezzi avrebbe infatti richiesto una complessa componente di **digital signal processing** e stima biomedica non centrale rispetto alle finalità dell'elaborato, che vertono primariamente sul **data engineering**, sulla modellazione dei database a documenti e sulle tecniche di **anomaly detection**.

L'indagine sperimentale si concentra su cinque metriche vitali primarie registrate dal modulo di monitoraggio `Solar8000`: il parametro `Solar8000/HR`, che quantifica la frequenza cardiaca espressa in battiti per minuto (bpm); il parametro `Solar8000/PLETH_SPO2`, che rileva la saturazione d'ossigeno dell'emoglobina periferica espressa in percentuale; il parametro `Solar8000/NIBP_SBP`, che misura la pressione arteriosa sistolica non invasiva espressa in millimetri di mercurio (mmHg); il parametro `Solar8000/NIBP_DBP`, che registra la corrispondente pressione arteriosa diastolica in millimetri di mercurio; il parametro `Solar8000/NIBP_MBP`, che esprime la pressione arteriosa media, grandezza fondamentale per stimare la perfusione d'organo.

Dal punto di vista fisiopatologico, la pressione arteriosa media non rappresenta una semplice media aritmetica, bensì una stima integrata nel ciclo cardiaco, approssimata dalla relazione clinica standard:

$$MBP \approx DBP + \frac{1}{3}(SBP - DBP)$$

Nel paziente chirurgico sottoposto ad anestesia generale, la vasodilatazione farmacologica e l'azione deprimente sul miocardio inducono frequentemente cali pressori improvvisi. Il mantenimento di un valore di $MBP \geq 65$ mmHg costituisce il target emodinamico raccomandato dalle linee guida internazionali per scongiurare danni ischemici acuti al circolo renale, coronarico e cerebrale.

L'eterogeneità temporale tra le sorgenti sensoristiche costituisce la sfida ingegneristica centrale del progetto: mentre la frequenza cardiaca e la saturazione periferica sono acquisite con cadenza continua a $1$ Hz tramite elettrodi ed emettitori ottici fotopletismografici, le misurazioni pressorie non invasive impiegano un bracciale pneumatico oscillometrico che necessita di tempi tecnici di gonfiaggio e sgonfiaggio compresi tra 20 e 40 secondi, ripetuti a intervalli programmati ogni due o cinque minuti. Ne deriva una matrice di serie storiche caratterizzata da forte asincronia e valori mancanti prolungati.

Queste rilevazioni fisiologiche sono affiancate da metadati clinici perioperatori scaricati contestualmente, tra cui l'età, il sesso e il reparto chirurgico di afferenza del paziente (`department`, ad esempio chirurgia generale, toracica o ginecologica). Il campione sperimentale selezionato comprende 50 interventi chirurgici completi per un totale di 311.173 secondi di tracciato attivo (corrispondenti a oltre 86 ore complessive di sala operatoria). L'integrazione di questi attributi contestuali risulta determinante non solo per consentire aggregazioni epidemiologiche, ma anche per guidare il partizionamento fisico dei dati e ottimizzare le strategie di indicizzazione all'interno della base di dati.

**Sintesi della coorte e delle sorgenti sensoristiche**

La base dati sperimentale raccoglie 50 interventi chirurgici per un totale di 311.173 secondi ($> 86$ ore di monitoraggio continuo). La telemetria integra segnali continui a 1 Hz da fotopletismografia ottica (frequenza cardiaca e saturazione d'ossigeno) con rilievi emodinamici intermittenti a bracciale oscillometrico (pressioni sistolica, diastolica e media), ancorando le soglie di allarme al limite critico di ipoperfusione d'organo

($MBP < 65$ mmHg).

## 1.3 Obiettivi e struttura dell'elaborato

Il lavoro svolto si articola attorno a cinque direttrici cardine che guidano l'intero ciclo di vita del dato clinico: la costruzione di una pipeline di raffinamento progressivo strutturata secondo il paradigma della **Medallion Architecture**, articolata nei livelli **Bronze**, **Silver** e **Gold**; l'archiviazione scalabile delle serie temporali mediante le **Time Series Collection** native di MongoDB 8.0, sfruttando il raggruppamento automatico a **bucket** compressi per massimizzare la densità dello storage e accelerare le query temporali; l'implementazione di un modello di **data governance** artigianale ispirato ai **data lakehouse**, comprendente un catalogo di registrazione delle serie con tracciamento della provenienza (**data lineage**), schemi formali di validazione JSON e l'impiego strutturato dei metadati per monitorare e preservare la qualità del dato; lo sviluppo di un'architettura multilivello di **anomaly detection**, capace di coniugare euristiche cliniche dominio-specifiche (lo **Shock Index** e la condizione di ipotensione severa) con modelli di **machine learning** non supervisionato (l'algoritmo ad alberi **Isolation Forest** e un **autoencoder** neurale ricorrente basato su PyTorch); l'ingegnerizzazione completa dell'ecosistema attraverso un backend REST realizzato con il framework **FastAPI**, una dashboard interattiva ricca di componenti visivi per la telemetria clinica e l'orchestrazione trasparente dei servizi mediante container Docker.

La trattazione è organizzata come segue: nel Capitolo 2 viene approfondita l'architettura della pipeline e il processo di pulizia e partizionamento nei livelli **Bronze** e **Silver**; nel Capitolo 3 vengono presentate l'archiviazione nel layer **Gold** su MongoDB, i benchmark di memorizzazione e l'intera impalcatura di **data governance**; nel Capitolo 4 sono esposti i modelli matematici e clinici di rilevamento delle anomalie con la relativa valutazione quantitativa; infine, nel Capitolo 5 sono descritti la progettazione del backend, l'interfaccia utente web, la containerizzazione con Docker e le prospettive di sviluppo futuro.

CAPITOLO 2

# Architettura della data pipeline

## 2.1 Il paradigma della medallion architecture

Nel trattamento di serie temporali complesse, l'adozione di un'architettura dati strutturata risulta indispensabile per assicurare la tracciabilità delle trasformazioni, l'integrità dei valori e la riproducibilità degli esperimenti. Nel presente progetto è stata implementata la **Medallion Architecture**, un pattern architetturale ampiamente diffuso nell'ingegneria dei **data lakehouse** che organizza il flusso di elaborazione in tre stadi logici a qualità progressivamente crescente, denominati convenzionalmente **Bronze**, **Silver** e **Gold**.

Come illustrato nella Figura 2.1, ciascun livello assolve a un compito ben definito all'interno della catena di raffinamento del dato: il livello **Bronze** accoglie i dati grezzi estratti direttamente dalla sorgente clinica, salvaguardandoli nella loro forma originaria senza alcuna alterazione; il livello **Silver** applica algoritmi di bonifica, normalizzazione, filtraggio degli artefatti sensoristici e partizionamento tematico per reparto; il livello **Gold** struttura i dati in documenti conformi al modello a serie storiche di MongoDB, predisponendoli al consumo efficiente da parte delle interfacce grafiche, dei servizi REST e dei modelli di **machine learning**.

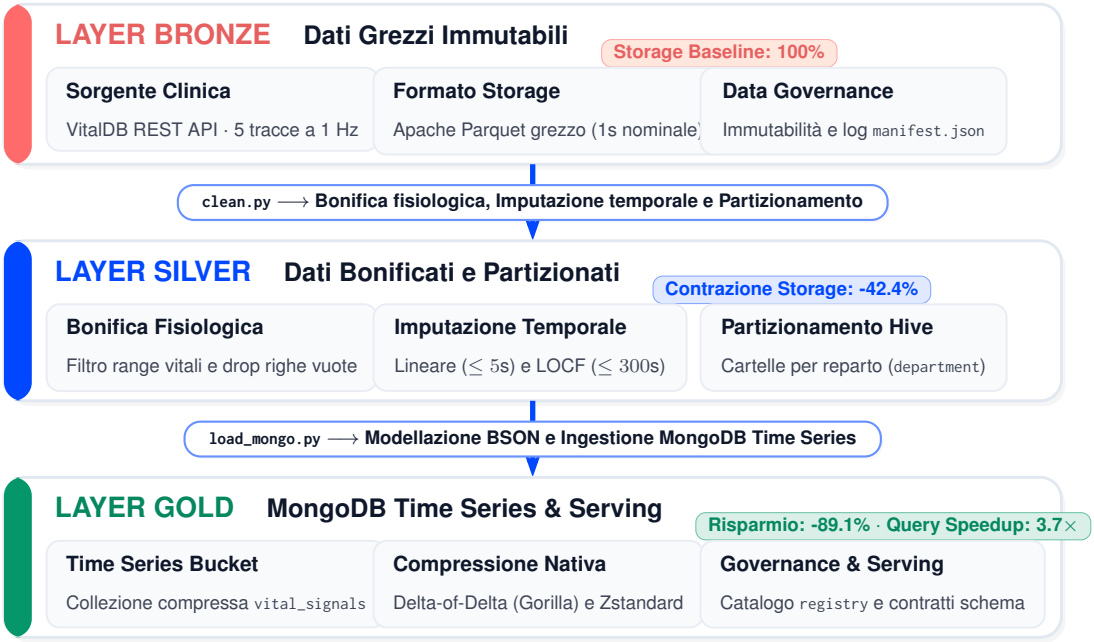



**Figura 2.1:** Panoramica dell'architettura Medallion e dei livelli Bronze, Silver e Gold generata in TikZ.

L'adozione di questa architettura multi-stadio garantisce che gli errori introdotti in un passaggio intermedio possano essere isolati e corretti rieseguendo le trasformazioni a partire dai livelli precedenti, preservando al contempo una netta separazione delle responsabilità tra ingestione, trasformazione analitica e fruizione finale del dato.

L'intera catena di elaborazione, schematizzata nella Figura 2.2, si articola in cinque fasi operative sequenziali che connettono l'estrazione remota alla visualizzazione telemetrica.

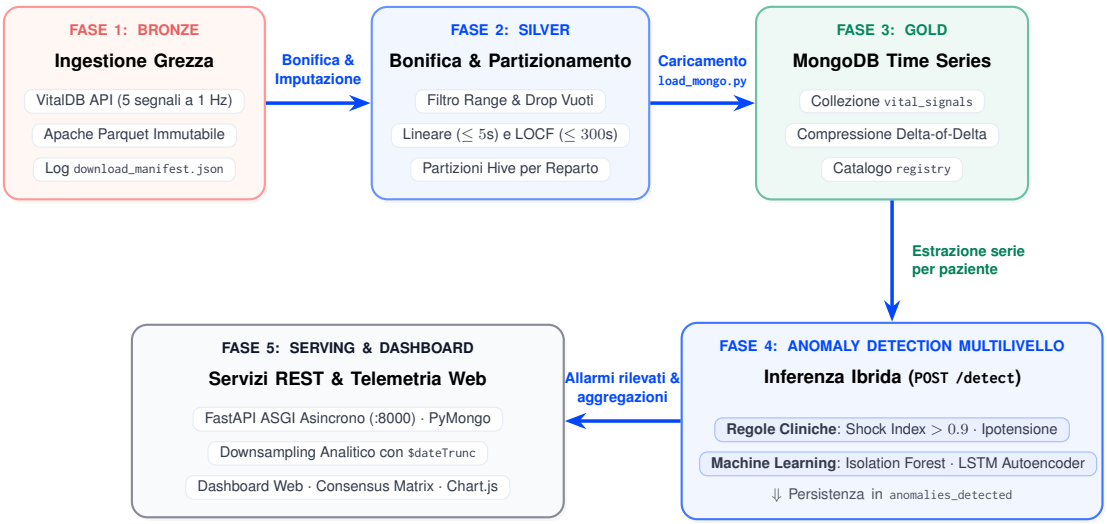



**Figura 2.2:** Architettura della pipeline di esecuzione end-to-end dall'ingestione alla visualizzazione.

## 2.2 Layer Bronze per l'ingestione e la storicizzazione dei dati grezzi

Il livello **Bronze** costituisce il punto di ingresso dell'intera infrastruttura. Il modulo di acquisizione, implementato nello script `src/bronze/download.py`, interagisce direttamente con l'archivio centrale di VitalDB sfruttando l'API Python ufficiale.

L'ingestione seleziona in modo automatico i casi clinici tramite la funzione `vitaldb.find_cases`, che individua i pazienti per i quali risultano registrate contemporaneamente le cinque metriche fisiologiche di interesse: la frequenza cardiaca, la saturazione d'ossigeno e le tre componenti pressorie. Per ciascun caso individuato, la procedura scarica i tracciati temporali mediante `vitaldb.load_case` impostando una frequenza nominale di campionamento pari a un secondo (`interval=1.0`).

I tracciati acquisiti vengono persistiti in locale nella cartella `data/bronze/` all'interno di file conformi allo standard Apache Parquet, con convenzione di denominazione `case_<ID>.parquet`. La scelta del formato Parquet per lo strato grezzo rispetto al testo semplice CSV è motivata dalla natura colonnare e compressa del formato, che riduce in misura rilevante lo spazio occupato su disco e memorizza in modo nativo i tipi di dato numerici a virgola mobile a 64 bit, scongiurando qualsiasi perdita di precisione numerica o ambiguità di codifica.

Contestualmente all'acquisizione dei segnali temporali, la procedura estrae le informazioni cliniche e perioperatorie mediante la funzione `load_clinical_data` resa disponibile dalla libreria di VitalDB, salvandole nel file ausiliario `data/bronze/clinical_data.parquet`. Questo file raccoglie attributi fondamentali quali l'età del paziente, il sesso, l'indice di massa corporea e il reparto chirurgico di ricovero.

In conformità con i principi di **data governance**, il livello **Bronze** adotta una politica di immutabilità rigorosa: i file vengono memorizzati esattamente nello stato in cui sono stati emessi dalla sorgente remota. Non viene applicata alcuna rimozione di valori nulli, né viene effettuata alcuna correzione di scala o troncamento temporale. Inoltre, per gestire con robustezza eventuali interruzioni di connettività durante il download su larga scala, la procedura aggiorna dinamicamente un file di registro `download_manifest.json`, consentendo alla pipeline di riprendere l'esecuzione dal punto esatto in cui era stata sospesa senza duplicare i trasferimenti di rete.

## 2.3 Layer Silver per la bonifica e il partizionamento dei dati

I dati clinici immagazzinati nel livello **Bronze** non possono essere impiegati direttamente per l'addestramento di modelli predittivi o per il monitoraggio in tempo reale a causa della notevole quantità di rumore, discontinuità e valori nulli. Il livello **Silver**, orchestrato dallo script `src/silver/clean.py`, si fa carico della bonifica del dato, della gestione dei valori mancanti, dell'etichettatura degli outlier fisiologici e della riorganizzazione fisica dei file su file system.

Il primo passo del processo consiste nell'eliminazione selettiva delle righe totalmente vuote. In molte fasi dell'intervento, ad esempio prima dell'ingresso effettivo del paziente in sala operatoria o dopo il distacco dei monitor, la registrazione produce timestamp privi di qualsiasi misurazione su tutte e cinque le tracce contemporaneamente. Queste righe non apportano alcun contenuto informativo e vengono rimosse per non appesantire inutilmente le strutture dati successive.

Successivamente, il modulo applica una strategia differenziata di imputazione per gestire la disomogeneità fisiologica delle sorgenti sensoristiche: per i segnali fisiologici continui, segnatamente la frequenza cardiaca (`Solar8000/HR`) e la saturazione periferica (`Solar8000/PLETH_SPO2`), le brevi perdite di segnale causate da artefatti di movimento vengono sanate mediante interpolazione lineare:

$$y(t) = y(t_0) + \frac{y(t_1) - y(t_0)}{t_1 - t_0}(t - t_0)$$

Per evitare di fabbricare andamenti biologici fittizi durante interruzioni prolungate del monitoraggio (quali il temporaneo distacco degli elettrodi o le interferenze da elettrobisturi), l'interpolazione viene rigorosamente circoscritta a intervalli di ampiezza non superiore a cinque secondi ($t_1 - t_0 \leq 5$ s). Questa scelta è motivata dalla fisiologia cardiovascolare: su scale temporali inferiori a cinque secondi la variazione di frequenza cardiaca è dominata da inerzia baroriflessa, rendendo l'ipotesi di linearità statisticamente plausibile; per i segnali pressori non invasivi (`NIBP_SBP`, `NIBP_DBP`, `NIBP_MBP`), la misurazione è intrinsecamente intermittente, avvenendo a intervalli discreti compresi tipicamente tra due e cinque minuti mediante il ciclo oscillometrico del bracciale. In questo scenario, l'interpolazione lineare risulterebbe priva di fondamento clinico, poiché creerebbe una fittizia transizione continua tra stati distanti nel tempo. Viene pertanto adottata la tecnica **Last Observation Carried Forward** (LOCF), propagando l'ultimo valore valido:

$$y(t) = y(t_{\text{last}}) \quad \forall t \in [t_{\text{last}}, t_{\text{last}} + 300 \text{ s}]$$

La propagazione è limitata a una finestra massima di 300 secondi (cinque minuti), oltre la quale la misura cessa di essere considerata clinicamente attendibile e viene lasciata indefinita.

A supporto della trasparenza analitica e dell'auditing clinico, il modulo calcola contestualmente la variabile temporale `nibp_age_s`:

$$\texttt{nibp_age_s} = t - t_{\text{last_cuff}}$$

Tale grandezza quantifica in ogni secondo l'età della misurazione pressoria corrente. Questo metadato si rivela determinante per il processo decisionale: qualora un algoritmo segnali un'ipotensione in presenza di un valore elevato di `nibp_age_s` (ad esempio superiore a 240 secondi), l'anestesista è tempestivamente informato che il dato riflette una misurazione remota e può disporre un gonfiaggio manuale del bracciale prima di intraprendere terapie farmacologiche aggressive.

La Figura 2.3 mette a confronto l'andamento del tracciato della frequenza cardiaca di un paziente reale prima e dopo l'intervento di bonifica. Nella parte superiore, il segnale grezzo estratto dal livello **Bronze** presenta evidenti salti e interruzioni causati da temporanee perdite di aderenza del sensore; nella parte inferiore, il segnale pulito del livello **Silver** si presenta continuo, omogeneo e pienamente utilizzabile per le analisi predittive.

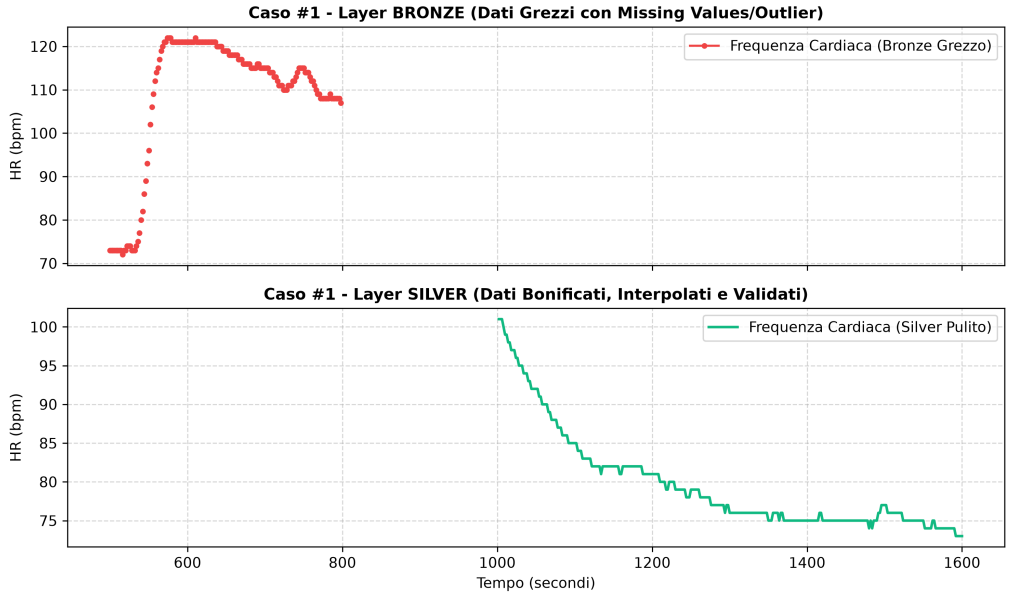



**Figura 2.3:** Confronto visivo tra il tracciato della frequenza cardiaca grezzo (Bronze) e bonificato (Silver).

Accanto all'imputazione dei valori mancanti, il livello **Silver** effettua un controllo di plausibilità fisiologica identificando i valori palesemente incompatibili con la sopravvivenza umana o attribuibili a malfunzionamenti hardware: frequenza cardiaca considerata valida tra $20$ e $250$ bpm; saturazione periferica considerata valida tra il $50\%$ e il $100\%$; pressione sistolica considerata valida tra $40$ e $250$ mmHg; pressione diastolica considerata valida tra $10$ e $180$ mmHg; pressione media considerata valida tra $20$ e $220$ mmHg.

I valori al di fuori di questi intervalli non vengono arbitrariamente cancellati, bensì etichettati tramite appositi flag booleani (`HR_outlier`, `SPO2_outlier`, `SBP_outlier`, `DBP_outlier`, `MBP_outlier`). In tal modo la pipeline conserva l'evidenza dell'anomalia rilevata a beneficio degli stadi successivi, senza alterare distruttivamente la serie storica.

L'implementazione concreta di queste trasformazioni è descritta nel Listato 2.1, estratto dal modulo `src/silver/clean.py`.

```
// Codice estratto da src/silver/clean.py
# 1. Filtra le righe completamente prive di segnali vitali
df_clean = df.dropna(subset=vital_tracks, how='all').copy()

# 2. Segnali continui (HR e SpO2): interpolazione lineare breve (max 5 sec)
cont_tracks = ['Solar8000/HR', 'Solar8000/PLETH_SPO2']
df_clean[cont_tracks] = df_clean[cont_tracks].interpolate(method='linear',
    limit=5)

# 3. Segnali emodinamici a intervalli (NIBP): LOCF con tetto di staleness a 300
    s
nibp_tracks = ['Solar8000/NIBP_SBP', 'Solar8000/NIBP_DBP', 'Solar8000/NIBP_MBP
    ']
if nibp_tracks:
    nibp_measured = df_clean[nibp_tracks[0]].notna()
    last_measure_time = df_clean['Time'].where(nibp_measured).ffill()
    df_clean['nibp_age_s'] = (df_clean['Time'] - last_measure_time).fillna
    (9999.0).astype(float)
    df_clean[nibp_tracks] = df_clean[nibp_tracks].ffill(limit=300)
```

```
16
17 # 4. Outlier fisiologici (etichettatura senza rimozione distruttiva)
18 df_clean['HR_outlier']   = (df_clean['Solar8000/HR'] < 20) | (df_clean['
       Solar8000/HR'] > 250)
19 df_clean['SPO2_outlier'] = (df_clean['Solar8000/PLETH_SPO2'] < 50) | (df_clean
       ['Solar8000/PLETH_SPO2'] > 100)
20 df_clean['MBP_outlier']  = (df_clean['Solar8000/NIBP_MBP'] < 20) | (df_clean['
       Solar8000/NIBP_MBP'] > 220)
```

**Listing 2.1:** Algoritmo di bonifica controllata, imputazione e tracciamento della freschezza temporale nel layer Silver.

**Sintesi di data governance nel layer Silver**

La separazione tra dati puliti e metadati di controllo assicura che il dato fisiologico non venga corrotto: l'imputazione lineare è ristretta alla scala del baroriflesso ($\leq 5$ s), il metodo LOCF è vincolato a un intervallo clinicamente credibile ($\leq 300$ s) con auditing esplicito della freschezza temporale tramite `nibp_age_s`, e le anomalie numeriche vengono tracciate tramite flag booleani senza alterazione distruttiva del dato.

Infine, i dati bonificati vengono salvati in formato Parquet all'interno di una gerarchia di directory partizionata fisicamente per reparto chirurgico di appartenenza (`department`):

`data/silver/department=<REPARTO>/case_<ID>.parquet`

Questa organizzazione ricalca il principio del **partition pruning** dei moderni **data lakehouse**: quando un'interrogazione o una procedura di addestramento richiede i dati relativi a una specifica specialità chirurgica, il lettore Apache Parquet ispeziona unicamente il sottoalbero della directory corrispondente senza scansionare l'intero archivio su disco, abbattendo le operazioni di I/O e velocizzando l'elaborazione.

## 2.4 Evoluzione del modello dati da tabella piatta a documento BSON

L'architettura della pipeline non si limita a innalzare la qualità intrinseca dei valori numerici, ma trasforma radicalmente il modello concettuale del dato nel passaggio verso il livello **Gold**.

Nei livelli **Bronze** e **Silver**, i dati risiedono sotto forma di tabelle piatte bidimensionali all'interno di file Parquet. Questo modello tabulare classico, pur risultando ideale per elaborazioni vettoriali in memoria tramite librerie quali `pandas`, mostra evidenti limiti nel contesto dell'archiviazione distribuita e del monitoraggio clinico in tempo reale: ogni riga ripete pedissequamente gli identificatori del paziente, non consente di esprimere relazioni gerarchiche native tra segnali e metadati, e richiede schemi rigidi poco adatti all'evoluzione dei parametri monitorati.

Nel livello **Gold**, il dato viene rimodellato come documento gerarchico BSON all'interno di MongoDB. La struttura del documento, mostrata nel Listato 2.2, è articolata in quattro sezioni logiche: il campo radice `timestamp`, che definisce l'ordinamento temporale ed è memorizzato come oggetto nativo `ISODate`; il sotto-documento `metadata`, che raccoglie in modo compatto tutte le proprietà immutabili del paziente e del sensore, tra cui l'identificativo del caso (`case_id`), il nome del monitor sorgente (`sensor_name`), il reparto chirurgico, l'età e il sesso; il sotto-documento `metrics`, che incapsula i valori numerici delle misurazioni fisiologiche correnti, opportunamente sanitizzati sostituendo i caratteri di barra con trattini bassi per evitare conflitti con la sintassi a percorsi di MongoDB; il sotto-documento `quality_flags`, che eredita i flag booleani generati nel livello **Silver**, preservando la qualifica di affidabilità di ciascuna misurazione.

```
// Documento gerarchico BSON archiviato nella collezione 'vital_signals'
{
  "_id": ObjectId("65d3f1a0e4b01a2b3c4d5e6f"),
  "timestamp": ISODate("2020-01-01T08:15:30Z"),
  "metadata": {
    "case_id": 1,
    "sensor_name": "Solar8000",
    "department": "General Surgery",
    "age": 64,
    "sex": "M"
  },
  "metrics": {
    "Solar8000_HR": 72.0,
    "Solar8000_PLETH_SPO2": 98.0,
    "Solar8000_NIBP_SBP": 120.0,
    "Solar8000_NIBP_DBP": 80.0,
    "Solar8000_NIBP_MBP": 93.3
  },
  "quality_flags": {
    "HR_outlier": false,
    "SPO2_outlier": false,
    "SBP_outlier": false,
    "DBP_outlier": false,
    "MBP_outlier": false
  }
}
```

**Listing 2.2:** Evoluzione del dato dal formato Parquet piatto al documento BSON nel layer Gold.

Questa rappresentazione semi-strutturata si rivela decisiva per sfruttare appieno le capacità del motore **Time Series** di MongoDB, poiché isola i metadati descrittivi dal flusso continuo delle metriche numeriche, abilitando strategie avanzate di compressione e indicizzazione.

CAPITOLO 3

# Archiviazione e governance con MongoDB Time Series

## 3.1 Layer Gold e archiviazione con MongoDB Time Series

Il livello **Gold** rappresenta lo strato di vertice dell'architettura dati, destinato ad alimentare in modo tempestivo e strutturato le interrogazioni analitiche, l'interfaccia grafica utente e gli algoritmi di rilevamento delle anomalie. I dati bonificati nel livello **Silver** vengono convogliati verso una base di dati MongoDB versione 8.0, all'interno della quale è configurata una collezione temporale nativa denominata `vital_signals`.

La funzionalità **Time Series Collection**, introdotta nelle recenti versioni di MongoDB, costituisce un'innovazione architetturale di rilievo rispetto alle collezioni a documenti tradizionali. In un database a documenti convenzionale, la memorizzazione di milioni di misurazioni ad alta frequenza comporterebbe un overhead insostenibile: ogni singolo secondo richiederebbe un documento BSON completo con il rispettivo identificatore univoco `_id`, le chiavi di campo replicate e le strutture di indice correlate. Nelle collezioni temporali, invece, MongoDB astrae la gestione fisica del dato raggruppando in modo del tutto trasparente le osservazioni temporali contigue all'interno di blocchi compressi denominati **bucket**.

La creazione della collezione `vital_signals`, implementata nello script `load_mongo.py`, richiede la definizione esplicita di tre parametri architetturali: il parametro `timeField`, impostato sul campo radice `timestamp`, che indica al motore il campo di riferimento temporale per ordinare e partizionare i record; il parametro `metaField`, impostato sul sotto-documento `metadata`, che raccoglie gli attributi descrittivi immutabili della serie associata a quel paziente; il parametro `granularity`, impostato al valore `seconds`, che suggerisce al motore la frequenza attesa delle misurazioni per ottimizzare la dimensione e la durata temporale di ciascun **bucket**.

All'interno dell'infrastruttura di archiviazione, MongoDB organizza i dati nella collezione di sistema interna `system.buckets.vital_signals`. In tale collezione, ogni singolo documento BSON non rappresenta una singola misurazione istantanea, bensì un intero **bucket** che racchiude un blocco compresso di campioni temporali consecutivi. Dal punto di vista strutturale, il documento di **bucket** si articola in tre componenti fondamentali: l'identificativo univoco `_id`, il sotto-documento `control` e il contenitore binario `data`. Il sotto-documento `control` ospita i metadati di controllo dell'intervallo, tra cui i valori estremi del timestamp (`min.timestamp` e `max.timestamp`), i valori minimi e massimi assunti dalle metriche numeriche all'interno del blocco, la versione dello schema interno (`version: 1`) e il contatore dei campioni (`count`), che può accogliere fino a 1.000 osservazioni temporali o estendersi fino al limite orario previsto dalla granularità definita.

Quando giungono nuovi campioni recanti i medesimi attributi immutabili (ossia lo stesso paziente e monitor), il motore inserisce i dati nel **bucket** aperto corrente fino al raggiungimento della capacità massima. Per comprimere le sequenze temporali all'interno della sezione `data`, MongoDB adotta una combinazione sinergica di algoritmi colonnari avanzati: l'asse temporale viene compresso mediante l'algoritmo **Delta-of-Delta**, originariamente introdotto nell'architettura Gorilla di Facebook per le serie storiche ad altissima frequenza. Indicando con $t_i$ il timestamp dell'osservazione corrente, la procedura calcola la differenza temporale di primo ordine rispetto

al campione precedente, $D_i = t_i - t_{i-1}$, e successivamente la variazione di secondo ordine rispetto all'incremento antecedente, $\Delta_i = D_i - D_{i-1}$. Nel contesto del monitoraggio intraoperatorio ad alta fedeltà a 1 Hz, i campioni si susseguono a intervalli rigorosamente costanti di un secondo: ne consegue che $D_i = 1$ s per ogni istante e, di riflesso, la derivata seconda temporale risulta identicamente nulla, $\Delta_i = 0$. In virtù delle regole di codifica bit-packing dell'algoritmo, il valore nullo viene serializzato impiegando un singolo bit `0`, invece degli 8 byte (64 bit) ordinariamente necessari per rappresentare un timestamp Unix standard a 64 bit o una data BSON. L'ingombro dell'ordinamento temporale crolla quindi a frazioni trascurabili di memoria; i vettori numerici delle metriche biometriche continue (quali frequenza cardiaca e saturazione) vengono codificati sfruttando la correlazione tra campioni consecutivi tramite differenziazione binaria XOR sulle mantisse in virgola mobile a 64 bit (standard IEEE 754), seguita da una compressione a blocchi con l'algoritmo **Zstandard** (zstd).

Un ulteriore beneficio architetturale di MongoDB rispetto al formato Parquet risiede nella gestione della sparsità e dei valori mancanti. Nel formato Parquet, ogni colonna numerica della tabella partizionata alloca necessariamente una maschera di bit di definizione (**definition level bitmask**) per tracciare per ciascuna riga la presenza o assenza del dato. Al contrario, all'interno della struttura flessibile BSON adottata da MongoDB, se un parametro vitale non viene misurato in un determinato secondo (come avviene tipicamente per la pressione non invasiva nei lunghi intervalli di riposo tra due cicli di gonfiaggio del bracciale), il relativo campo viene semplicemente omesso dal sotto-documento `metrics`. Questa assenza non comporta alcuna allocazione di byte né padding numerico, massimizzando la densità informativa dello storage.

## 3.2 Benchmark prestazionale e di occupazione dello storage

Per valutare in modo quantitativo e scientificamente rigoroso i benefici apportati dalla transizione dai file Parquet alla collezione temporale di MongoDB, è stata eseguita una campagna di benchmark automatizzata mediante lo script `benchmark_etl.py`.

La misurazione ha preso in esame un campione a larga scala di 50 casi clinici completi, per un totale di 311.173 punti temporali bonificati. La sperimentazione ha indagato tre parametri cardine: l'occupazione fisica dello storage su disco, la latenza di scansione delle interrogazioni temporali e la completezza delle celle numeriche. I risultati ottenuti sono compendiati nella Tabella 3.1 e illustrati graficamente nella Figura 3.1.

| **Parametro / Metrica** | **Bronze (grezzo)** | **Silver (pulito Parquet)** | **Gold (MongoDB TS)** |
|---|---|---|---|
| Occupazione disco (50 casi) | 3.88 MB | 2.24 MB (-42.4%) | 0.42 MB (-89.1%) |
| Latenza scansione singolo caso | 13.48 ms (Parquet) | 13.48 ms (Parquet) | 3.63 ms (Pushdown indicizzato) |
| Fattore di velocizzazione | 1.0× (Riferimento) | 1.0× | **3.7× più veloce** |
| Valori mancanti misurati | 67.6% | 32.6% (post-LOCF) | 0.0% (Omissione BSON) |

**Tabella 3.1:** Confronto quantitativo di storage, latenza e qualità dati tra i layer su 311.173 record.

Come emerge dai dati riportati nella Tabella 3.1, il passaggio dai file Parquet non compressi del livello **Bronze** (3.88 MB) al formato bonificato del livello **Silver** (2.24 MB) produce una prima contrazione del volume del 42.4%, dovuta alla rimozione delle righe completamente prive di segnale vitale. L'ulteriore transizione al livello **Gold** su MongoDB abbatte lo spazio occupato a soli 0.42 MB, facendo registrare un risparmio complessivo di storage dell'89.1% rispetto al dato grezzo originario. Questo risultato dimostra l'eccellente efficienza dell'algoritmo di compressione a **bucket** per serie temporali biometriche a elevata frequenza.

Sul fronte delle prestazioni computazionali, il protocollo di test ha adottato una metodologia comparativa uniforme: è stata misurata la mediana dei tempi di risposta su quindici iterazioni

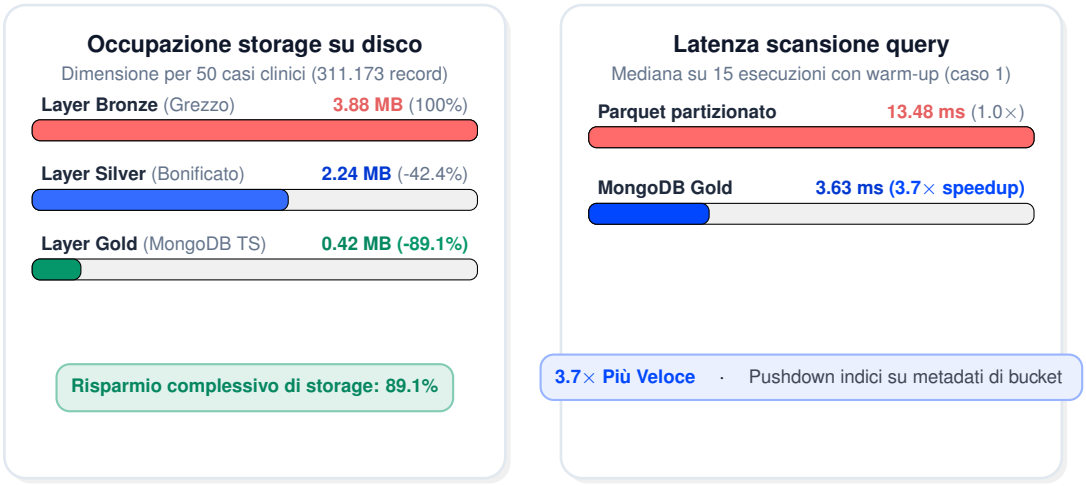



**Figura 3.1:** Benchmark comparativo dell'occupazione disco e della latenza di query generato in TikZ.

successive relative al caso clinico 1, precedute da una fase di riscaldamento (**warmup**) per stabilizzare le cache del sistema operativo e del database. L'interrogazione mirava a estrarre la media della frequenza cardiaca per l'intero arco temporale dell'intervento. La lettura del file Parquet partizionato ha richiesto una latenza mediana di 13.48 ms, mentre la corrispondente query di aggregazione in MongoDB, eseguita tramite pipeline con operatore `$match` e `$group`, è stata evasa in appena 3.63 ms. Il motore di MongoDB consegue dunque un fattore di velocizzazione pari a 3.7 volte, beneficiando del filtraggio anticipato a livello di metadati del **bucket** prima di procedere alla decodifica delle serie numeriche.

**Vantaggi architetturali di MongoDB Time Series**

La combinazione della compressione a **bucket** con l'algoritmo **Delta-of-Delta** consente di abbattere lo storage su disco dell'89.1% rispetto ai dati grezzi **Bronze**. Contestualmente, il **query pushdown** sui metadati indicizzati del **bucket** esclude a priori le serie temporali dei pazienti non richiesti, velocizzando la scansione analitica di 3.7 volte rispetto all'accesso su file Parquet.

## 3.3 Data governance artigianale in ottica data lakehouse

Un aspetto centrale del progetto, concordato e approfondito durante le fasi di revisione con la relatrice, risiede nella formalizzazione della **data governance**. Nei moderni contesti industriali, l'adozione di un'architettura a livelli richiede meccanismi stringenti per garantire l'integrità strutturale, il tracciamento della provenienza e l'affidabilità delle transazioni. Piattaforme dedicate quali Delta Lake, Apache Iceberg o Apache Hudi forniscono queste garanzie mediante registri di transazioni distribuiti (**transaction log**) e cataloghi integrati all'interno di motori di calcolo analitico quali Spark o Trino.

Nel presente lavoro, per mantenere l'architettura compatta, autosufficiente ed eseguibile in ambienti ospedalieri locali senza dipendenze computazionali onerose, si è scelto di emulare artigianalmente le proprietà fondamentali di un **data lakehouse** sfruttando esclusivamente gli strumenti e le primitive native di MongoDB. Tale impostazione poggia su quattro componenti complementari: il catalogo di registrazione delle serie, la tabella dei metadati descrittivi, l'applicazione di schemi formali di validazione e una strategia applicativa di compensazione per le scritture.

### 3.3.1 Catalogo e registrazione della provenienza (collezione registry)

Per tenere traccia dello stato di ingestione di ciascun caso clinico e garantire la ricostruibilità storica del processo di caricamento, è stata istituita la collezione `registry`. Tale collezione funge da catalogo centrale e opera come l'equivalente di un metastore applicativo.

Ogni qualvolta una serie temporale viene promossa dal livello **Silver** al livello **Gold**, viene inserito nel registro un documento che attesta l'avvenuta pubblicazione. Come mostrato nel Listato 3.1, il documento registra l'identificativo univoco del caso clinico (`case_id`), il numero complessivo di punti temporali validati e immessi nella serie (`record_count`), la versione formale dello schema logico adottato (`schema_version`) e l'oggetto di provenienza (`provenance`), che riporta la fase di elaborazione (`silver_to_gold`) e l'istante temporale esatto di caricamento con marcatura UTC.

```
// Validazione applicata alla collezione 'registry' in MongoDB
db.runCommand({
  collMod: "registry",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["case_id", "schema_version", "provenance"],
      properties: {
        case_id: {
          bsonType: ["int", "long"],
          description: "ID del caso clinico, intero a 32 o 64 bit obbligatorio"
        },
        schema_version: {
          bsonType: "string",
          description: "Versione dello schema, stringa obbligatoria"
        },
        record_count: {
          bsonType: ["int", "long"],
          description: "Numero complessivo di record temporali caricati"
        },
        provenance: {
          bsonType: "object",
          required: ["step", "timestamp"],
          properties: {
            step: { bsonType: "string" },
            timestamp: { bsonType: "date" }
          }
        }
      }
    }
  },
  validationLevel: "strict",
  validationAction: "error"
});
```

**Listing 3.1:** Schema formale di validazione JSON Schema per la collezione registry.

La collezione `registry` è protetta da regole native di **JSON Schema Validation** configurate con livello rigoroso (`validationLevel: "strict"`) e azione di rifiuto immediato (`validationAction: "error"`). In questo modo, qualsiasi tentativo di inserire nel catalogo record incompleti o privi dei vincoli di provenienza viene categoricamente respinto dal motore del database.

### 3.3.2 Metadati descrittivi delle tracce (collezione track_metadata)

Accanto al catalogo delle esecuzioni, la governance necessita di una definizione semantica chiara del significato di ciascuna variabile misurata. A questo scopo è stata creata la collezione ausiliaria `track_metadata`, inizializzata tramite lo script `mongo/registry_schema.js`.

Per ciascuna metrica fisiologica registrata dal monitor `Solar8000`, la collezione documenta il nome formale della traccia, la versione del contratto dati, l'unità di misura fisica adottata (`bpm`, percentuale o `mmHg`) e l'intervallo numerico fisiologicamente plausibile (`expected_range`). Questa collezione funge da vero e proprio contratto di interfaccia tra il personale sanitario, i moduli di bonifica e i consumatori a valle, chiarendo in modo inequivocabile i limiti attesi di ogni segnale.

### 3.3.3 Limiti transazionali e strategia applicativa di compensazione

Un'analisi rigorosa della piattaforma impone di evidenziare con trasparenza i vincoli intrinseci dell'infrastruttura scelta. Come documentato nel manuale ufficiale di MongoDB, le **Time Series Collection** non consentono operazioni di inserimento all'interno di transazioni multi-documento distribuite; l'esecuzione di una scrittura temporale sotto una sessione transazionale genera l'errore nativo 263 (`OperationNotSupportedInTransaction`). Ne consegue che non è tecnicamente possibile effettuare un'operazione atomica ACID congiunta tra la scrittura fisica delle serie in `vital_signals` e l'aggiornamento del catalogo in `registry`.

Per ovviare a tale limite senza compromettere la coerenza dei dati, nel modulo di caricamento `load_mongo.py` è stata adottata una strategia applicativa di compensazione ordinata: la procedura esegue dapprima l'inserimento in blocco di tutti i punti della serie all'interno della collezione `vital_signals`; solo a fronte del completamento con esito positivo dell'inserimento dei dati temporali, la procedura procede alla registrazione del caso nel catalogo `registry`; qualora la prima operazione dovesse sollevare un errore di rete o di I/O, il controllo interrompe l'esecuzione e il documento di catalogo non viene scritto. In questo modo si impedisce l'insorgenza di casi fantasma all'interno del catalogo. Al contempo, per rendere l'operazione idempotente e prevenire duplicazioni accidentali in caso di riavvio della procedura, il caricatore verifica preliminarmente la presenza del caso nel `registry`: se il caso risulta già catalogato, il caricamento dei dati associati viene saltato a priori.

## 3.4 Pipeline di aggregazione analitica e ottimizzazione delle query

La disponibilità dei dati ad alta frequenza nel livello **Gold** consente di sviluppare pipeline di interrogazione analitica avanzate per alimentare in tempo reale la visualizzazione grafica e gli algoritmi diagnostici. Per evitare che la complessità computazionale si trasferisca sull'applicazione web o sull'infrastruttura di rete, le elaborazioni intensive di filtraggio, raggruppamento e aggregazione temporale vengono demandate direttamente al motore interno di MongoDB, sfruttando il framework di aggregazione nativo (**Aggregation Pipeline**).

### 3.4.1 Downsampling temporale efficiente lato database con $dateTrunc

Nel monitoraggio intraoperatorio continuo, un singolo intervento chirurgico di media o lunga durata può protrarsi per diverse ore, accumulando decine di migliaia di punti temporali per paziente (oltre 21.000 campioni per una procedura di sei ore). Il trasferimento indiscriminato dell'intero tracciato a frequenza di 1 Hz dal database al browser del client comporterebbe un sovraccarico insostenibile: il payload JSON della risposta HTTP supererebbe decine di megabyte, saturando la banda di rete e rallentando il motore di rendering grafico JavaScript dell'interfaccia utente.

Per risolvere questa criticità, nel modulo `src/analysis/queries.py` è stata sviluppata la funzione `downsample_case`, richiamabile dall'API specificando il parametro `window_seconds`. La funzione ese-

gue una riduzione controllata della risoluzione temporale (**downsampling**) interamente a livello di base di dati mediante una pipeline strutturata su tre stadi consecutivi, come illustrato nel Listato 3.2: lo stadio `$match` seleziona esclusivamente i documenti pertinenti al paziente richiesto sfruttando il campo indicizzato `metadata.case_id`; lo stadio `$group` impiega l'operatore temporale `$dateTrunc` per troncare e raggruppare i timestamp delle misurazioni in intervalli discreti di ampiezza prefissata (ad esempio 30, 60 o 300 secondi), specificando l'unità di misura temporale (`unit: "second"`) e la dimensione del contenitore (`binSize: window_seconds`). Contestualmente, per ciascuna metrica vitale presente nel sotto-documento `metrics`, il gruppo calcola la media aritmetica dei valori osservati nella finestra mediante l'accumulatore `$avg`; lo stadio `$sort` ordina cronologicamente le finestre aggregate garantendo la continuità sequenziale del flusso restituito.

```
// Aggregazione per il downsampling temporale in MongoDB
db.vital_signals.aggregate([
  // 1. Filtraggio per caso clinico (sfrutta l'indice sui metadati)
  { $match: { "metadata.case_id": case_id } },

  // 2. Raggruppamento temporale a finestre fisse con media delle metriche
  {
    $group: {
      _id: {
        $dateTrunc: {
          date: "$timestamp",
          unit: "second",
          binSize: NumberInt(window_seconds)
        }
      },
      HR:       { $avg: "$metrics.Solar8000_HR" },
      PLETH_SPO2: { $avg: "$metrics.Solar8000_PLETH_SPO2" },
      NIBP_SBP: { $avg: "$metrics.Solar8000_NIBP_SBP" },
      NIBP_DBP: { $avg: "$metrics.Solar8000_NIBP_DBP" },
      NIBP_MBP: { $avg: "$metrics.Solar8000_NIBP_MBP" }
    }
  },

  // 3. Ordinamento cronologico delle finestre aggregate
  { $sort: { _id: 1 } }
]);
```

**Listing 3.2:** Pipeline di aggregazione MongoDB per il downsampling temporale con l'operatore $dateTrunc.

Questa strategia garantisce che il volume dei dati trasmessi sulla rete si riduca di un fattore esattamente proporzionale all'ampiezza della finestra scelta (ad esempio una contrazione di 60 volte per finestre da un minuto), consentendo alla dashboard web di caricare e visualizzare tracciati chirurgici di svariate ore con latenze inferiori a cento millisecondi.

### 3.4.2 Ottimizzazione dell'accesso anagrafico ed eliminazione del problema N+1

Un'ulteriore ottimizzazione architetturale riguarda l'estrazione dell'elenco dei pazienti per la barra di navigazione della dashboard, implementata nella rotta REST `GET /cases`.

In un disegno non ottimizzato, l'applicazione interrogherebbe preliminarmente la collezione `registry` per recuperare l'elenco dei casi censiti, per poi iterare su ciascuno di essi ed eseguire una query distinta (`find_one`) sulla collezione `vital_signals` allo scopo di recuperare i metadati di contesto del paziente (reparto chirurgico, età e sesso). Questo approccio ingenuo genera il noto proble-

ma delle $N + 1$ query: per una coorte di $N$ pazienti, il sistema invierebbe al database $N + 1$ interrogazioni separate, moltiplicando i tempi di latenza di rete e il carico di parsing del motore.

Nel presente sistema, il problema è stato risolto strutturando una procedura a due sole operazioni indipendenti dal numero di pazienti della coorte: la prima lettura interroga in blocco la collezione `registry` per prelevare lo stato di caricamento e i conteggi certificati; la seconda operazione invia a `vital_signals` una pipeline di aggregazione con un singolo stadio `$group` raggruppato per `$metadata.case_id`, che estrae istantaneamente il primo documento associato a ciascun paziente mediante l'operatore `$first` sui campi `department`, `age` e `sex`.

Le due strutture informative vengono poi unite in memoria all'interno del backend Python in un singolo dizionario. In questo modo il numero di interazioni con il database rimane rigorosamente fisso a due, azzerando qualsiasi sovraccarico al crescere della dimensione del dataset.

## 3.5 Uso dei metadati per la data quality e analisi critica

Nel disegno dell'architettura, i metadati svolgono una duplice funzione: abilitano prestazioni di accesso ottimali e governano l'osservabilità della qualità del dato lungo tutte le fasi della pipeline.

Il campo `metaField` della collezione temporale svolge un ruolo tecnologico primario: raccogliendo in modo strutturato l'identificativo del caso, il monitor di provenienza, il reparto chirurgico, l'età e il sesso del paziente, MongoDB genera automaticamente indici compositi che accoppiano i metadati al timestamp. Le interrogazioni analitiche che filtrano per paziente o per reparto beneficiano del cosiddetto **query pushdown**: il motore individua ed esamina esclusivamente i **bucket** pertinenti senza dover decodificare le serie storiche dei pazienti estranei alla query.

Inoltre, per garantire che le valutazioni di qualità effettuate nel livello **Silver** non vadano disperse nella successiva fase di analisi o addestramento dei modelli, ciascun punto memorizzato nel livello **Gold** incorpora l'oggetto `quality_flags`. In tal modo, un algoritmo di **machine learning** o un clinico possono interrogare la serie distinguendo immediatamente le misurazioni fisiologicamente conformi da quelle che avevano evidenziato valori anomali durante i controlli di plausibilità.

A supporto della tracciabilità complessiva, la conclusione della fase di pulizia **Silver** comporta la generazione automatica del file `quality_report.json`, che censisce per ogni singolo paziente il numero di campioni originali, i campioni mantenuti, le righe rimosse, il tasso di valori nulli residui e il conteggio analitico degli outlier rilevati per ciascun canale. Al fine di mantenere memoria storica dell'evoluzione della qualità, la pipeline provvede ad archiviare la copia precedente del report all'interno della cartella `quality_history` assegnando una marca temporale univoca.

A fronte di questi risultati, è doveroso svolgere alcune considerazioni critiche sulla semantica delle metriche rilevate: la completezza dichiarata pari a zero valori nulli nel livello **Gold** (come evidenziato nella Tabella 3.1) è la conseguenza diretta della struttura documentale BSON e dell'eliminazione preventiva delle righe totalmente vuote. In MongoDB, i campi che non presentano una misura valida vengono omessi dal sotto-documento `metrics` per non occupare spazio; tale assenza non deve essere confusa con un recupero matematico miracoloso di tutti i dati mancanti originali, ma rappresenta un'ottimizzazione rappresentativa del dato; il processo di bonifica nel livello **Silver** elimina 333.464 righe su 644.637 campioni complessivi presenti nel **Bronze**. Questa operazione abbatte drasticamente il rumore e dimezza il tasso di celle nulle (dal 67.6% al 32.6%), ma comporta la perdita di oltre la metà dei timestamp iniziali, corrispondenti alle fasi in cui i sensori non registravano segnali utili; l'asse temporale è stato normalizzato sommando i secondi relativi dell'intervento a una data convenzionale fissa (`2020-01-01 + Time`). Sebbene tale espediente risponda pienamente alle esigenze di ordinamento e indicizzazione delle collezioni

temporali, in una futura implementazione ospedaliera di produzione sarà opportuno raccordare i campioni all'orario effettivo registrato nel cartellino anestesiologico.

CAPITOLO 4

# Modelli di anomaly detection

## 4.1 Approccio ibrido e multilivello al rilevamento di anomalie

Nel monitoraggio intraoperatorio continuo, l'identificazione precoce di deviazioni fisiologiche critiche costituisce un obiettivo primario per la sicurezza del paziente. La dinamica biologica di un organismo sottoposto ad anestesia generale e a stimoli chirurgici è tuttavia estremamente complessa ed eterogenea: gli eventi avversi possono manifestarsi sia sotto forma di repentine cadute pressorie sia come graduali e insidiose dissociazioni tra parametri vitali multipli.

Per intercettare questo ampio spettro di condizioni, il presente progetto non si affida a un'unica tecnica di analisi, bensì implementa un approccio ibrido multilivello fondato su quattro metodologie complementari: due regole cliniche deterministiche dominio-specifiche, formulate sulla base delle evidenze della letteratura medica (lo **Shock Index** e la condizione di ipotensione severa); due algoritmi di **machine learning** non supervisionato capaci di operare sullo spazio congiunto delle serie storiche, segnatamente l'algoritmo ad alberi **Isolation Forest** e un modello neurale ricorrente basato su **LSTM Autoencoder**.

La complementarità di questo schema risponde a una precisa motivazione clinica e ingegneristica: le regole cliniche hanno una solida giustificazione fisiopatologica e forniscono riscontri trasparenti e immediatamente comprensibili dal personale medico, ma falliscono nell'identificare anomalie multivariate sfumate che precedono l'evento conclamato; al contrario, i modelli di **machine learning** non supervisionato eccellono nel riconoscere correlazioni inattese e configurazioni atipiche ad alta dimensionalità, sebbene necessitino di una contestualizzazione per distinguere gli artefatti statistici dalle vere emergenze cliniche.

## 4.2 Regole cliniche dominio-specifiche

Le regole cliniche implementate nello script `src/detection/detector.py` operano direttamente sulle grandezze fisiologiche archiviate nel livello **Gold**, valutando soglie note nella pratica anestesiologica.

### 4.2.1 Shock Index clinico

Lo **Shock Index** ($SI$) è un indicatore emodinamico adimensionale ampiamente utilizzato nei reparti di rianimazione e pronto soccorso per la valutazione tempestiva del rischio di shock ipovolemico, emorragico o cardiogeno. Esso è definito matematicamente come il rapporto tra la frequenza cardiaca ($HR$) e la pressione arteriosa sistolica ($SBP$):

$$SI = \frac{\text{Solar8000_HR}}{\text{Solar8000_NIBP_SBP}}$$

In un soggetto adulto sano in condizioni stazionarie di riposo, il valore normale dello **Shock Index** oscilla tra $0.5$ e $0.7$. Quando la gittata sistolica cala a seguito di un'emorragia o di una vasodilatazione indotta dai farmaci anestetici, il sistema nervoso autonomo tenta di compensare aumentando la frequenza cardiaca (tachicardia riflessa) mentre la pressione sistolica tende a diminuire. Il modulo contrassegna come anomalo qualsiasi istante temporale in cui si verifichi la

condizione:

$$SI > 0.9$$

Il superamento di questa soglia testimonia il fallimento dei meccanismi fisiologici di compenso e segnala un'instabilità emodinamica imminente che richiede un'immediata infusione di fluidi o l'infusione di vasopressori.

### 4.2.2 Ipotensione severa

La seconda euristica clinica mira a identificare stati di grave compromissione perfusiva e ipossia cellulare. La regola valuta la compresenza simultanea di una pressione arteriosa media insufficiente e di una desaturazione d'ossigeno periferico:

$$\text{Solar8000_NIBP_MBP} < 65 \text{ mmHg} \quad \wedge \quad \text{Solar8000_PLETH_SPO2} < 90\%$$

Il mantenimento di una pressione arteriosa media ($MBP$) al di sopra di $65$ mmHg è universalmente riconosciuto nelle linee guida cliniche come la soglia minima necessaria per garantire l'autoregolazione del flusso ematico nei distretti cerebrale e renale. Quando tale ipoperfusione si associa a una caduta della saturazione al di sotto del $90\%$, l'organismo entra in una condizione di grave sofferenza ipossico-ischemica. La coincidenza temporale dei due segnali costituisce un allarme di massima priorità in sala operatoria.

## 4.3 Ingegnerizzazione delle feature temporali

Per consentire agli algoritmi di **machine learning** di catturare non soltanto i valori istantanei dei parametri ma anche la dinamica evolutiva e la tendenza temporale delle serie biometriche, la procedura esegue una fase di estrazione di feature temporali derivate.

A partire dal segnale della frequenza cardiaca (`Solar8000/HR`), che presenta la cadenza di campionamento più continua e regolare a frequenza di 1 Hz, vengono calcolate tre feature dinamiche fondamentali. Indicando con $x_t$ il valore della frequenza cardiaca all'istante temporale $t$ (in secondi) e con $W = 5$ campioni l'ampiezza della finestra temporale retrospettiva, le metriche vengono definite formalmente come: la media mobile a cinque campioni (`HR_rolling_mean`), che attenua le micro-oscillazioni e gli artefatti ad alta frequenza senza introdurre ritardi significativi:

$$\mu_t = \frac{1}{W} \sum_{k=0}^{W-1} x_{t-k}$$

la deviazione standard mobile a cinque campioni (`HR_rolling_std`), che quantifica la dispersione e l'instabilità istantanea del battito cardiaco nel brevissimo termine:

$$\sigma_t = \sqrt{\frac{1}{W-1} \sum_{k=0}^{W-1} (x_{t-k} - \mu_t)^2}$$

la differenza prima (`HR_delta`), che misura la velocità istantanea di accelerazione o decelerazione del ritmo cardiaco:

$$\Delta x_t = x_t - x_{t-1}$$

L'introduzione di $\Delta x_t$ riveste un ruolo fisiologico cruciale: una repentina variazione positiva di $\Delta x_t$ (oltre 10–15 bpm in un singolo secondo) riflette una scarica adrenergica massiva indotta da un risveglio intraoperatorio o da uno stimolo algico chirurgico acuto, mentre un crollo improvviso testimonia un riflesso vagale o un blocco atrioventricolare.

Al fine di garantire l'assoluta correttezza metodologica, il calcolo di queste finestre mobili viene vincolato al raggruppamento per singolo paziente mediante la trasformazione `df.groupby("case_id")`. Questo accorgimento impedisce rigorosamente che i confini temporali di un caso clinico si sovrappongano a quelli del caso successivo, scongiurando qualsiasi contaminazione analitica tra pazienti differenti nei punti di transizione.

Il vettore complessivo delle feature impiegato dai modelli di **machine learning** comprende otto grandezze numeriche: le cinque metriche vitali originarie (frequenza cardiaca, saturazione d'ossigeno e le tre componenti pressorie) affiancate dalle tre feature dinamiche derivate sulla frequenza. Al contrario, la variabile `nibp_age_s` calcolata nel livello **Silver** viene deliberatamente esclusa dal vettore; essa rappresenta infatti un metadato di governance attestante la freschezza temporale della misura e non una proprietà biologica intrinseca del paziente.

## 4.4 Modelli di machine learning non supervisionato

L'approccio basato sull'apprendimento non supervisionato prescinde dalla presenza di etichette a priori e mira a identificare pattern anomali nello spazio delle otto feature biometriche.

### 4.4.1 Isolation Forest

Il primo algoritmo impiegato è l'**Isolation Forest**, un metodo di ensemble basato su alberi decisionali isolanti (**iTree**) particolarmente efficace per serie storiche ad alta dimensionalità.

Il principio teorico dell'algoritmo sfrutta le proprietà geometriche delle anomalie: i punti anomali sono pochi rispetto alla popolazione complessiva e tendono a posizionarsi in regioni periferiche e poco dense dello spazio vettoriale. L'algoritmo partiziona ricorsivamente lo spazio campionario selezionando casualmente una feature ed effettuando un taglio uniforme tra il valore minimo e massimo osservato. Poiché i campioni normali sono densamente raggruppati, essi richiedono un numero elevato di partizioni per essere isolati all'interno di una foglia; al contrario, le istanze anomale, trovandosi isolate nello spazio, vengono separate con pochissimi tagli decisionali.

La lunghezza media del percorso (**path length**) dalla radice alla foglia attraverso la foresta di alberi fornisce una misura inversa dello score di anomalia. Nello script di produzione, l'algoritmo viene istanziato con una quota di contaminazione teorica fissata al 5% (`contamination=0.05`) e con un seme deterministico fissato a 42 per garantire la riproducibilità dei risultati.

### 4.4.2 LSTM Autoencoder neurale (PyTorch)

Per catturare le interdipendenze sequenziali e la memoria a lungo termine dei segnali fisiologici, è stato progettato e implementato un modello di deep learning basato su reti ricorrenti: l'**LSTM Autoencoder**, realizzato mediante la libreria PyTorch.

Come schematizzato nella Figura 4.1, l'architettura elabora le serie storiche mediante una struttura simmetrica di tipo **encoder-decoder**: il modello riceve in ingresso un tensore tridimensionale di forma $(B, W_s, C)$, dove $B$ rappresenta la dimensione del batch, $W_s = 30$ campioni (corrispondenti a 30 secondi di telemetria continua) è l'ampiezza della finestra scorrevole e $C = 8$ è il numero di canali biometrici standardizzati con punteggio z; l'encoder è costituito da uno strato ricorrente LSTM a 32 unità nascoste (`hidden_size=32`), che riceve la sequenza temporale $X \in \mathbb{R}^{30\times 8}$ e la comprime progressivamente nel vettore di stato nascosto finale $h_{\text{enc}} \in \mathbb{R}^{32}$. Tale vettore costituisce il **context vector** compresso, che riassume l'intera dinamica evolutiva del blocco temporale; per alimentare il decoder, il vettore $h_{\text{enc}}$ viene replicato per 30 passi temporali consecutivi (**repeat vector**), fornendo a ciascun istante del processo di ricostruzione la medesima rappresentazione globale del contesto; il decoder è composto da un secondo strato ricorrente LSTM speculare a 32 unità, che elabora la sequenza riprodotta generando gli stati di

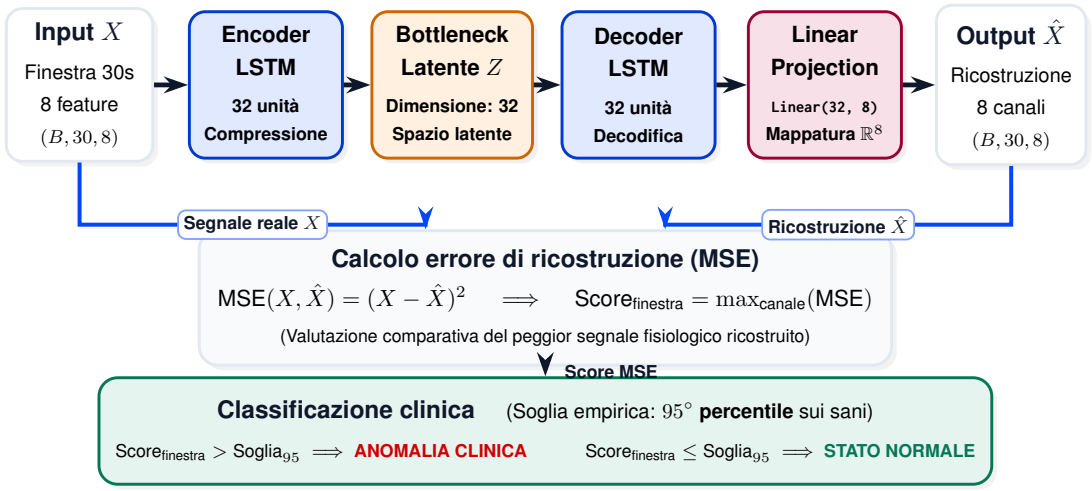



**Figura 4.1:** Architettura vettoriale della rete neurale LSTM Autoencoder con proiettore lineare in TikZ.

decodifica $h_t^{\text{dec}} \in \mathbb{R}^{32}$ per ciascuno dei 30 istanti; un layer di proiezione lineare finale (`nn.Linear`), posizionato all'uscita del decoder, proietta ciascuno stato nascosto nello spazio continuo originale delle feature: $\hat{X}_t = W_p h_t^{\text{dec}} + b_p$, con pesi $W_p \in \mathbb{R}^{8\times32}$ e bias $b_p \in \mathbb{R}^8$. L'introduzione di questo strato lineare si è rivelata determinante: la funzione di attivazione tangente iperbolica nativa delle celle LSTM limita infatti le uscite all'intervallo ristretto $[-1, 1]$, impedendo la corretta ricostruzione di variabili standardizzate con punteggio z che superano tali estremi. Il proiettore lineare elimina questo vincolo consentendo la ricostruzione sull'intero asse reale.

L'architettura della rete in PyTorch, definita mediante la funzione interna `_make_lstm_autoencoder_class` nel modulo `detector.py`, è riportata nel Listato 4.1.

```
// Modulo PyTorch implementato in src/detection/detector.py
class LSTMAutoencoder(nn.Module):
    def __init__(self, n_features=8, hidden_size=32, n_layers=1):
        super().__init__()
        # Encoder: comprime la sequenza (B, 30, 8) nello stato nascosto (B, 32)
        self.encoder = nn.LSTM(n_features, hidden_size, n_layers, batch_first=
    True)
        # Decoder: ricostruisce la sequenza temporale partendo dal vettore
    latente
        self.decoder = nn.LSTM(hidden_size, hidden_size, n_layers, batch_first=
    True)
        # Proiettore lineare: mappa lo stato nascosto R^32 nello spazio
    originale R^8
        self.out = nn.Linear(hidden_size, n_features)

    def forward(self, x):
        # x ha forma (batch_size, sequence_length=30, n_features=8)
        _, (hidden, _) = self.encoder(x)
        # Replicazione del context vector per ciascuno dei 30 timestep
        context = hidden.permute(1, 0, 2).repeat(1, x.size(1), 1)
        output, _ = self.decoder(context)
        return self.out(output)  # dimensione ricostruita: (batch, 30, 8)
```

**Listing 4.1:** Definizione del modello PyTorch LSTM Autoencoder con proiettore lineare per l'anomaly detection.

La preparazione dei dati per la rete neurale adotta una rigida politica di segmentazione per

paziente mediante la classe `LSTMAutoencoderDetector`: i dati vengono raggruppati per `case_id` e standardizzati con scala z-score; la suddivisione delle serie in finestre da 30 secondi viene eseguita mediante la funzione vettoriale senza copia `np.lib.stride_tricks.sliding_window_view`. Questa operazione genera le viste sulle finestre temporali manipolando unicamente gli offset dei puntatori di memoria (**strides**), senza allocare o duplicare copie fisiche dell'array in memoria RAM. In questo modo è possibile processare agevolmente oltre 300.000 finestre con un consumo di memoria estremamente contenuto; il modello viene addestrato con ottimizzatore Adam (tasso di apprendimento $10^{-3}$) e funzione di perdita di errore quadratico medio (MSE) per dieci epoche su un campione rappresentativo uniforme di finestre.

Durante la fase di inferenza, la rete valuta la discrepanza tra la finestra di ingresso $X$ e la corrispondente finestra ricostruita $\hat{X}$. Nel calcolo dell'errore di ricostruzione complessivo della finestra, si è deliberatamente scelto di non utilizzare la media aritmetica dell'errore quadratico medio su tutti gli otto canali. In ambito emodinamico, la mediazione globale introdurrebbe infatti un grave rischio di diluizione: se sette parametri su otto rimangono stabili, una caduta acuta e drammatica di un singolo canale vitale (quale una grave desaturazione $SpO_2$ al di sotto dell'80% per ostruzione respiratoria improvvisa) verrebbe attenuata dall'effetto mediato degli altri segnali, non riuscendo a superare la soglia di allarme. Per scongiurare tale evenienza, l'errore della finestra viene definito come il massimo errore quadratico medio osservato tra i canali biometrici:

$$\mathcal{E}(X) = \max_{c \in \{1,\dots,C\}} \left( \frac{1}{W_s} \sum_{t=1}^{W_s} (X_{t,c} - \hat{X}_{t,c})^2 \right)$$

L'errore così ottenuto viene assegnato con precisione all'ultimo istante temporale della finestra scorrevole.

La soglia decisionale di anomalia viene fissata in modo non parametrico al $95^\circ$ percentile della distribuzione empirica degli errori $\mathcal{E}(X)$ su tutte le finestre. I campioni il cui errore di ricostruzione supera tale soglia vengono classificati come anomalie fisiologiche multivariate complesse.

**Principio di sensibilità emodinamica nel deep learning**

L'impiego del massimo errore quadratico medio canale per canale ($\max_c \text{MSE}_c$) garantisce che il modello neurale mantenga una sensibilità elevata anche di fronte a collassi isolati di un singolo parametro vitale (es. ipossia acuta o crollo pressorio), impedendo che la stabilità contemporanea degli altri canali fisiologici disinneschi l'allarme multivariato.

## 4.5 Valutazione empirica, metriche e concordanza

La valutazione delle prestazioni dei modelli di **machine learning** in ambito clinico su serie storiche ad alta frequenza incontra un ostacolo metodologico significativo: l'assenza di etichette reali validate istante per istante da un anestesista umano.

Per condurre una valutazione quantitativa rigorosa, nello script `evaluate_models.py` è stato adottato un protocollo di concordanza clinica: l'unione logica degli eventi critici intercettati dalle regole cliniche dominio-specifiche (**Shock Index** superiore a 0.9 oppure ipotensione severa) è stata assunta come riferimento di **pseudo-ground truth**. Su un totale di 311.173 punti temporali esaminati relativi ai 50 casi clinici, sono stati censiti 15.240 istanti di emergenza clinica reale, corrispondenti a una prevalenza positiva di base (**base rate**) pari al 4.90%.

Questa marcata asimmetria tra le classi impone una riflessione critica su una trappola statistica classica, nota come **Accuracy Paradox**: un classificatore banale che predica sistematicamente la classe maggioritaria (rispondendo invariabilmente "normale" per ogni secondo dell'intervento)

ottiene un'accuratezza globale pari al 95.10%, pur risultando totalmente inutile ai fini clinici avendo fallito nell'individuare anche una sola emergenza. Ne consegue che l'accuratezza globale non rappresenta una metrica informativa per la valutazione del sistema. La sperimentazione deve focalizzarsi rigorosamente sulle metriche orientate alla sola classe anomala, quali la precision, la recall, il punteggio f1 e l'area sottesa alla curva Precision-Recall (PR-AUC). I risultati ottenuti sono compendiati nella Tabella 4.1.

| Modello / Algoritmo | Precision (anomalia) | Recall (anomalia) | F1-score (anomalia) | PR-AUC (score continuo) |
|---|---|---|---|---|
| Classificatore banale (Baseline) | 0.000 | 0.000 | 0.000 | 0.049 |
| Isolation Forest | **0.140** | **0.143** | **0.141** | **0.075** |
| LSTM Autoencoder | 0.083 | 0.085 | 0.084 | 0.063 |

**Tabella 4.1:** Valutazione comparativa dei modelli ML rispetto allo standard clinico su 311.173 record.

Come si evince dalla Tabella 4.1, entrambi gli algoritmi di **machine learning** superano nettamente la baseline casuale (pari alla prevalenza positiva del 4.90%), conseguendo valori di PR-AUC pari a 0.075 per l'**Isolation Forest** e 0.063 per l'**Autoencoder**. I valori assoluti apparentemente contenuti di Precision ed F1-score riflettono fedelmente la natura dell'apprendimento non supervisionato e meritano un'attenta disamina fisiopatologica.

I modelli di **machine learning** non sono stati addestrati tramite regressione supervisionata per ricalcare meccanicamente la formula algebrica dello Shock Index o la soglia statica dell'ipotensione severa, bensì per rilevare deviazioni multivariabili atipiche all'interno dell'intero spazio a otto dimensioni. Ne discende che i campioni segnalati dagli algoritmi non supervisionati che non trovano corrispondenza nella **pseudo-ground truth** non devono essere semplicisticamente etichettati come "falsi positivi" o errori modellistici. Al contrario, un'analisi retrospettiva delle serie dimostra che molti di questi allarmi intercettano manifestazioni cliniche sub-conclamate di grande rilevanza: micro-instabilità del ritmo cardiaco caratterizzate da improvvisi aumenti della deviazione standard mobile (`HR_rolling_std`), correlabili a extrasistoli ventricolari o fibrillazioni atriali parossistiche; rapide decelerazioni del battito cardiaco ($\Delta x_t < -15$ bpm) indotte da riflessi vagali conseguenti a trazioni peritoneali o manipolazioni chirurgiche dei visceri; transitori cali pressori in cui la pressione arteriosa media scende repentinamente verso valori di confine (ad esempio da 95 a 68 mmHg), senza ancora varcare la soglia critica di 65 mmHg ma testimoniando comunque un'alterazione acuta del tono vascolare periferico.

In questa prospettiva, l'approccio multilivello dimostra tutto il proprio valore pratico: le regole cliniche deterministiche operano come paracadute di sicurezza trasparente per le emergenze manifeste, mentre gli algoritmi di **machine learning** estendono l'orizzonte temporale di allerta cogliendo le sottili avvisaglie prodromiche dell'instabilità emodinamica prima che essa si trasformi in uno shock conclamato.

L'efficacia della modellazione neurale trova conferma nell'analisi della distribuzione dell'errore di ricostruzione dell'**Autoencoder**, illustrata nella Figura 4.2. La quasi totalità dei punti temporali fisiologicamente stabili si concentra in una campana ristretta con valori di errore quadratico medio prossimi allo zero; al contrario, le configurazioni atipiche generano una marcata coda lunga a destra della distribuzione, superando nettamente la soglia empirica del $95^{\circ}$ percentile evidenziata dalla linea tratteggiata.

La reciproca interazione tra le quattro metodologie di detection è visualizzata nella matrice di concordanza e correlazione di Figura 4.3. Il grafico evidenzia la concordanza positiva esistente tra le regole emodinamiche e i modelli non supervisionati, confermando come l'approccio ibrido multilivello consenta di intersecare la specificità dell'euristica medica con l'ampiezza di copertura del **machine learning**.

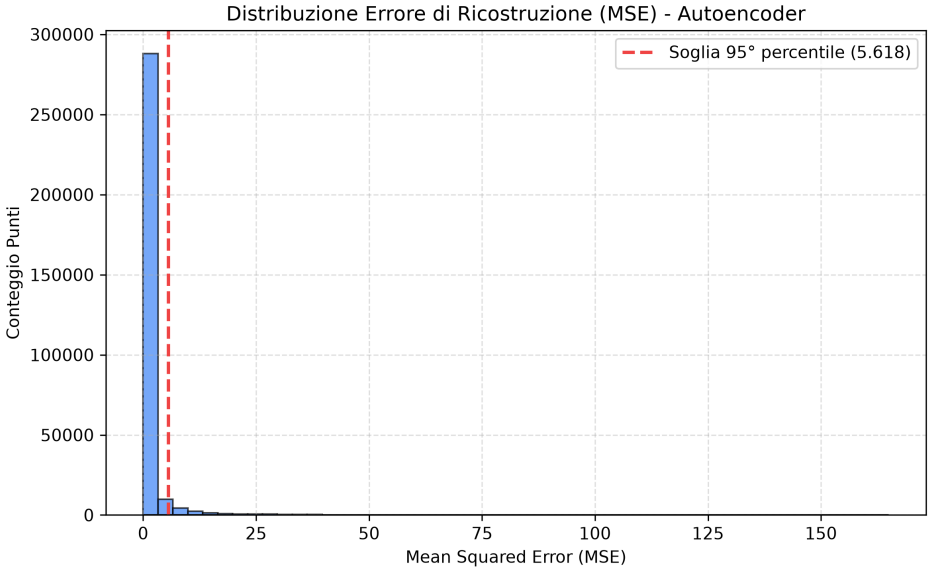



**Figura 4.2:** Distribuzione dell'errore di ricostruzione (MSE) dell'autoencoder con soglia al $95^{\circ}$ percentile.

## 4.6 Persistenza e tracciamento delle anomalie su MongoDB

A completamento del flusso di rilevamento, gli istanti temporali valutati come anomali da almeno una delle quattro metodologie vengono persistiti all'interno della collezione MongoDB denominata `anomalies_detected`.

Ciascun documento memorizzato nella collezione raccoglie le seguenti informazioni: l'identificativo del caso clinico (`case_id`); il timestamp esatto in cui si è verificata l'anomalia; l'array `methods`, che elenca i nomi degli algoritmi che hanno segnalato l'istante (ad esempio `["shock_index", "isolation_forest"]`); la marcatura temporale di esecuzione `detected_at`, che garantisce la tracciabilità dell'elaborazione analitica.

Al fine di garantire l'idempotenza della pipeline e prevenire la presenza di anomalie obsolete nel database, la procedura nello script `detector.py` cancella preliminarmente tutti i riscontri pregressi associati al caso clinico esaminato prima di registrare i nuovi risultati. Tale accorgimento assicura che, qualora una successiva esecuzione su un paziente stabilizzato non rilevi alcuna anomalia, le segnalazioni storiche vengano rimosse correttamente senza lasciare falsi allarmi nella base di dati.

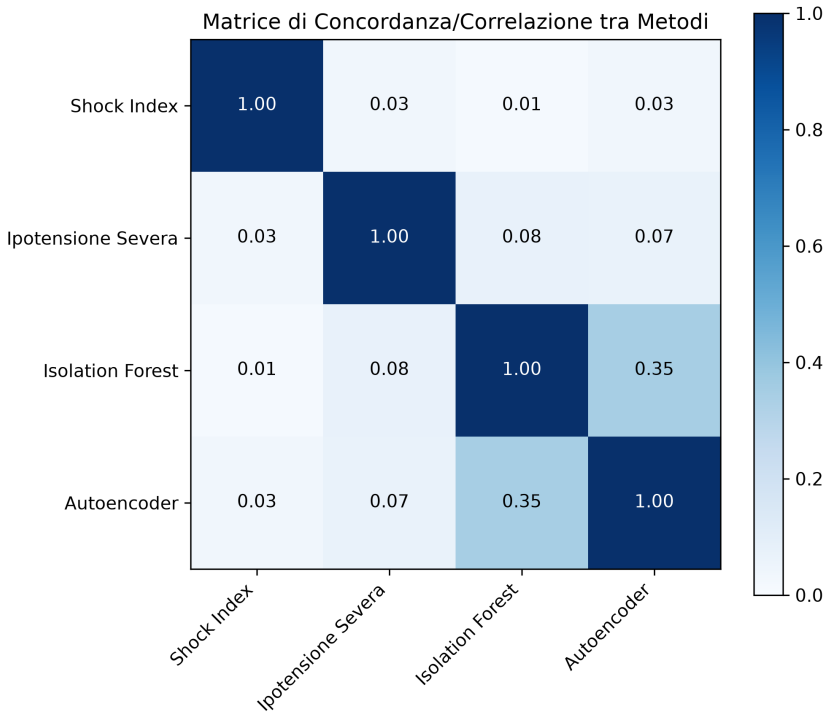



**Figura 4.3:** Matrice di concordanza/correlazione tra le regole cliniche e i modelli ML.

CAPITOLO 5

# Ingegnerizzazione del sistema, interfaccia grafica e conclusioni

## 5.1 Backend REST API con FastAPI

L'integrazione e la fruizione dei dati archiviati nel livello **Gold** di MongoDB e l'esecuzione degli algoritmi di rilevamento delle anomalie sono orchestrate da un backend sviluppato in linguaggio Python mediante il framework **FastAPI**.

La scelta di FastAPI è motivata dalle elevate prestazioni del suo motore asincrono basato su **Starlette** e **Pydantic**, che consentono la validazione rigorosa dei parametri di richiesta, la serializzazione nativa dei documenti BSON in formato JSON e la generazione automatica della documentazione interattiva OpenAPI (disponibile all'indirizzo `/docs`). Il servizio REST opera all'interno del container applicativo `vitaldb_api` sulla porta di rete 8000 ed è implementato nel file `api/main.py`.

Dal punto di vista dell'efficienza delle risorse, il backend gestisce la connettività verso MongoDB mediante un'unica istanza globale del client `pymongo.MongoClient`. Questo accorgimento implementa un pool di connessioni riutilizzabili (**connection pooling**) che mantiene aperti i socket TCP tra le varie richieste HTTP, azzerando il costo di negoziazione dell'handshake di rete per ciascuna chiamata all'API. Inoltre, prima della trasmissione dei dati sulla rete, il backend esegue una bonifica dei valori numerici: i valori non validi o mancanti (`np.nan`) vengono convertiti nel valore Python `None`, serializzato nel corrispondente valore `null` standard dello standard JSON. Questa sanitizzazione previene la generazione di token non standard `NaN`, che provocherebbero errori di decodifica bloccanti nei parser JavaScript dei browser moderni (`JSON.parse`).

L'interfaccia espone un insieme organico di rotte operative: l'endpoint `GET /health`, che svolge la funzione di monitoraggio dello stato di salute del container ed effettua un'interrogazione di ping al database. Esso restituisce un oggetto strutturato contenente lo stato della connettività e le statistiche aggregate di sistema: il numero di casi censiti nel `registry`, il totale dei punti temporali archiviati in `vital_signals` e il conteggio cumulativo delle anomalie salvate in `anomalies_detected`; l'endpoint `GET /cases`, che estrae l'elenco dei casi clinici registrati nel catalogo. Per prevenire il classico degrado prestazionale dovuto al problema delle $N + 1$ query (in cui per ogni caso verrebbe effettuata una lettura distinta per recuperare reparto, età e sesso del paziente), il metodo adotta un'unica pipeline di aggregazione MongoDB con operatori `$group` e `$first`. Questa ottimizzazione riduce l'intero recupero anagrafico a due sole operazioni complessive sul database; l'endpoint `GET /cases/{case_id}/series`, che restituisce la serie temporale biometrica del paziente richiesto. L'endpoint supporta il parametro opzionale `window_seconds` (con valori selezionabili quali 30, 60 o 300 secondi): quando specificato, l'API attiva la pipeline di aggregazione temporale descritta nel capitolo precedente con operatore `$dateTrunc`, riducendo drasticamente il payload di rete trasferito al browser per tracciati che si estendono per molte ore; l'endpoint `POST /cases/{case_id}/detect`, la cui implementazione è descritta nel Listato 5.1, innesca l'esecuzione on-demand dell'intera pipeline di **anomaly detection** sul paziente selezionato. L'elaborazione calcola contestualmente le regole cliniche dello **Shock Index** e dell'ipotensione severa, unitamente alle inferenze dei modelli **Isolation Forest** e **Autoencoder**. I riscontri anomali vengono

persistiti nella collezione `anomalies_detected` e restituiti al client suddivisi per metodologia; gli endpoint dedicati alla trasparenza e alla governance, segnatamente `GET /quality`, `GET /benchmark` e `GET /evaluation`, che espongono direttamente in formato JSON i report di bonifica del livello **Silver**, i risultati del benchmark di storage e latenza, e le metriche quantitative di validazione dei modelli di **machine learning**.

```
// Endpoint implementato nel backend FastAPI (api/main.py)
@app.post("/cases/{case_id}/detect")
def detect_case(case_id: int):
    """Esegue la pipeline di anomaly detection multilivello sul caso specificato."""
    db = get_db()
    df = get_case_series(db, case_id)
    if df.empty:
        raise HTTPException(status_code=404, detail=f"Nessun dato per il caso #{case_id}")

    # Esegue regole cliniche (Shock Index, Ipotensione Severa) ed algoritmi ML
    anomalies_df = run_detection_pipeline(df, case_id)

    # Persistenza atomica dei riscontri nella collezione MongoDB 'anomalies_detected'
    db['anomalies_detected'].delete_many({"case_id": case_id})
    if not anomalies_df.empty:
        records = anomalies_df.to_dict(orient="records")
        db['anomalies_detected'].insert_many(records)

    return {
        "status": "ok",
        "case_id": case_id,
        "total_anomalies": len(anomalies_df),
        "methods_summary": {
            "shock_index": int(anomalies_df['shock_index_anomaly'].sum()),
            "severe_hypotension": int(anomalies_df['severe_hypotension_anomaly'].sum()),
            "isolation_forest": int(anomalies_df['isolation_forest_anomaly'].sum()),
            "autoencoder": int(anomalies_df['autoencoder_anomaly'].sum())
        }
    }
```

**Listing 5.1:** Definizione della rotta REST per l'esecuzione della pipeline di anomaly detection in FastAPI.

Accanto agli endpoint analitici, FastAPI ospita due montaggi di file statici: la rotta `/dashboard`, che distribuisce l'intera interfaccia web interattiva, e la rotta `/presentazione`, che rende fruibili le slide illustrative del progetto per la discussione accademica.

## 5.2 Dashboard web e interfaccia utente interattiva

A corredo dell'infrastruttura dati è stata sviluppata un'interfaccia grafica web concepita per consentire al personale sanitario o all'analista di esplorare i tracciati biometrici, analizzare le anomalie e ispezionare i parametri di governance.

Come mostrato nella Figura 5.1, l'interfaccia è stata realizzata seguendo i principi di un'estetica moderna e funzionale. Dal punto di vista architetturale, la dashboard rifugge la complessità e la pesantezza di framework monolitici, impiegando standard aperti del web: un'architettura




**Figura 5.1:** Panoramica dell'interfaccia grafica web della dashboard per il monitoraggio e la governance.

in html5 semantico, fogli di stile css3 con variabili a token per la gestione dinamica del tema chiaro e scuro (**light/dark mode**), e codice in javascript puro. L'aspetto tipografico è valorizzato da caratteri tipografici dedicati, tra cui **Figtree** per le intestazioni, **Space Grotesk** per i testi di interfaccia e **JetBrains Mono** per i valori numerici e i timestamp.

Per garantire modularità e manutenibilità senza richiedere strumenti di compilazione esterni quali Webpack o Vite, il codice dell'interfaccia adotta un sistema di caricamento modulare asincrono implementato nello script `dashboard/js/loader.js`. Il file principale `index.html` definisce dei contenitori dichiarativi denominati slot mediante l'attributo personalizzato `data-include` (ad esempio `<div data-include="partials/sidebar.html"></div>`). All'avvio dell'applicazione, la funzione `loadPartials` individua tutti gli slot mediante `querySelectorAll`, invia richieste HTTP parallele tramite `Promise.all` e l'API `fetch`, e provvede a sostituire ciascun tag con il contenuto HTML recuperato. Al fine di impedire che il browser mantenga in cache versioni obsolete dei file durante gli aggiornamenti, il caricatore accoda a ciascuna richiesta un parametro di versione esplicito (`?v=3.8`), realizzando una tecnica efficace di invalidazione della cache (**cache busting**).

La piattaforma si articola su due ambienti di lavoro principali, accessibili tramite selettore in testata: la vista di telemetria clinica e la vista di governance del **lakehouse**.

### 5.2.1 Vista di telemetria clinica e anomaly detection

Questo spazio di lavoro è dedicato al monitoraggio del paziente e alla rilevazione delle instabilità. L'ambiente si compone dei seguenti moduli integrati: la barra laterale (**sidebar**), che elenca tutti i pazienti caricati nel livello **Gold** fornendo un campo di ricerca rapida e badge identificativi con reparto chirurgico, età e sesso; il **Patient HUD** (Heads-Up Display), un banner informativo collocato in cima alla pagina che visualizza l'identificativo del paziente selezionato, il reparto e i parametri anagrafici. Il modulo ospita i comandi operativi principali: il selettore della risoluzione temporale (che consente di visualizzare il dato grezzo a 1 secondo oppure aggregazioni a 30 secondi, 60 secondi o 5 minuti), il pulsante di ricarica e il pulsante per avviare il calcolo delle anomalie; la **Vitals Grid**, una griglia a cinque tessere informative progettata secondo lo stile **bento-grid**. Ciascuna tessera ospita un indicatore visivo pulsante, il valore medio corrente e il range fisiologico di riferimento per la frequenza cardiaca ($60 - 100$ bpm), la saturazione d'ossigeno ($95 - 100\%$), la pressione arteriosa con valore medio ($> 65$ mmHg), lo **Shock Index** e il contatore cumulativo degli eventi anomali rilevati; la **Consensus Matrix**, definita nel file `partials/consensus-matrix.html`, che implementa un pannello interattivo a quattro schede corrispondenti ai quattro metodi di rilevamento (Shock Index, ipotensione severa, Isolation Forest e Autoencoder). Ciascuna scheda mostra un badge con il conteggio in tempo reale degli eventi individuati e consente di cliccare per isolare ed evidenziare sui grafici esclusivamente le anomalie prodotte da quel metodo specifico. Inoltre, ciascuna scheda integra un pulsante informativo che apre una finestra modale con la spiegazione fisiologica, la formula analitica e la soglia di allarme adottata; i tracciati telemetrici realizzati con la libreria **Chart.js**: due grafici interattivi sincronizzati ad asse temporale comune. Il grafico superiore traccia l'andamento congiunto di frequenza cardiaca e saturazione, mentre il grafico inferiore visualizza le tre componenti pressorie (sistolica, diastolica e media). I due grafici condividono la traccia di mira (**crosshair**) e i comandi di ingrandimento temporale, consentendo una correlazione visiva immediata tra alterazioni del ritmo e variazioni pressorie. Quando viene attivato il rilevamento, i punti anomali vengono evidenziati mediante marcatori cromatici sovrapposti alle curve dei segnali; la tabella analitica delle anomalie, collocata a fondo pagina, che elenca cronologicamente ogni singolo secondo in cui è stata riscontrata un'irregolarità, riportando i valori biometrici istantanei, lo Shock Index calcolato e l'elenco degli algoritmi concordanti.

### 5.2.2 Vista di data lakehouse, governance e benchmark

La seconda schermata dell'interfaccia visualizza in modo trasparente le metriche di architettura dati e qualità del dato sincronizzate in tempo reale tramite le API REST: due schede comparative sull'ottimizzazione dello storage e della latenza, che illustrano tramite barre grafiche la contrazione progressiva dello spazio su disco dal livello **Bronze** al livello **Gold** (evidenziando la riduzione dell'89.1%) e il fattore di accelerazione di 3.7 volte ottenuto con il **pushdown** degli indici MongoDB; la tabella di valutazione dei modelli di **machine learning**, sincronizzata dall'endpoint `GET /evaluation`, che presenta il confronto tra **Isolation Forest** e **LSTM Autoencoder** esponendo le metriche di Precision, Recall, F1-score e PR-AUC calcolate su 311.173 punti; la tabella del **Data Quality and Governance Report**, alimentata dall'endpoint `GET /quality`, che elenca analiticamente per ciascun caso clinico i campioni grezzi del **Bronze**, i campioni bonificati nel **Silver**, le righe eliminate e il conteggio degli outlier fisiologici rilevati per ciascun parametro vitale.

## 5.3 Containerizzazione ed orchestrazione con Docker

Al fine di garantire l'assoluta riproducibilità dell'ambiente software, l'indipendenza dal sistema operativo ospitante e la rapidità di dispiegamento in contesti ospedalieri, l'intero sistema è stato ingegnerizzato e containerizzato mediante Docker e Docker Compose.

L'intera architettura multi-livello del sistema, illustrata nella Figura 5.2, connette in modo organico l'interfaccia client, il gateway applicativo e il cluster di archiviazione MongoDB.

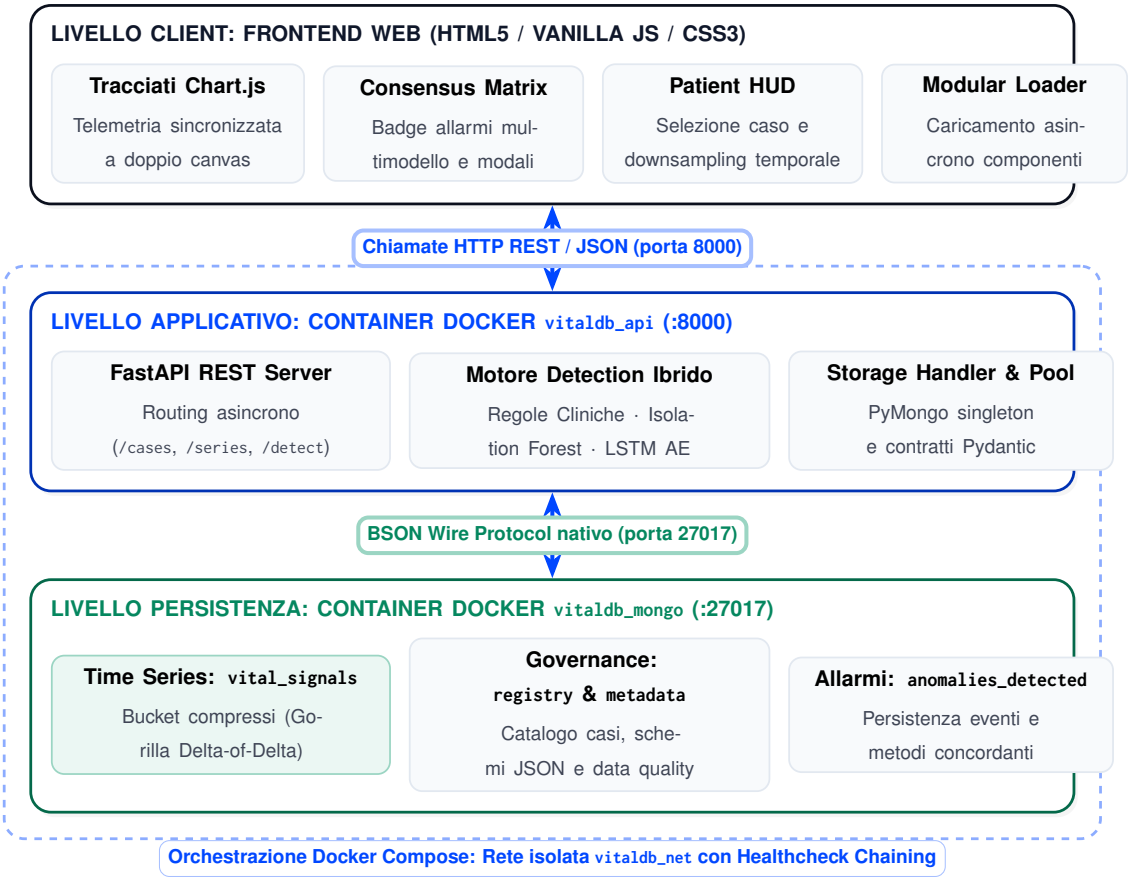



**Figura 5.2:** Architettura multi-livello completa dell'applicazione e containerizzazione Docker.

L'infrastruttura definita nel file `docker-compose.yml` orchestra due container interconnessi su una rete virtuale dedicata denominata `vitaldb_net`: il container del database `vitaldb_mongo`, basato sull'immagine ufficiale di MongoDB 8.0. Il servizio viene avviato configurando automaticamente un Replica Set a singolo nodo (`-replSet rs0`), prerequisito architetturale per abilitare le funzionalità transazionali native di MongoDB. L'integrità dello storage è garantita da un volume persistente su disco, mentre le collezioni, gli schemi JSON e i metadati iniziali vengono predisposti automaticamente all'avvio mediante gli script montati nella directory di inizializzazione `/docker-entrypoint-initdb.d/`; il container applicativo `vitaldb_api`, costruito a partire da un'immagine leggera basata su Python 3.12-slim. Il container incorpora l'interprete dell'API FastAPI, il server ASGI `uvicorn` e l'insieme delle librerie matematiche necessarie all'esecuzione dei modelli, tra cui `pymongo`, `scikit-learn`, `pandas`, `numpy` e la versione per CPU di `torch`.

Un elemento architetturale di rilievo risiede nell'orchestrazione delle dipendenze basata su **healthcheck** attivi. Poiché l'API richiede che il database non sia soltanto attivo a livello di processo ma abbia completato con successo l'inizializzazione del Replica Set e il caricamento degli schemi, il servizio `mongo` esegue un controllo ciclico mediante il client nativo `mongosh`. Lo script verifica che il nodo appartenga allo stato operativo primario o, in caso contrario, ne forza l'inizializzazione (`rs.initiate`). Nel file di composizione, il container `api` dichiara la dipendenza:

```
depends_on: { mongo: { condition: service_healthy } }
```

Questo meccanismo impedisce che il backend entri in esecuzione prima che il database sia pienamente operativo, azzerando qualsiasi fallimento di connessione all'avvio dello stack. L'intero

sistema può quindi essere generato e avviato con un singolo comando:

```
docker compose up -build -d
```

**Isolamento infrastrutturale e resilienza del deployment**

L'orchestrazione Docker garantisce il disaccoppiamento totale tra il calcolo analitico e la persistenza dei dati: il volume permanente preserva le serie storiche e il catalogo di governance a fronte di riavvii del container, mentre la rete isolata `vitaldb_net` e il meccanismo di healthcheck chaining proteggono l'integrità delle transazioni durante l'avvio a freddo dello stack.

## 5.4 Conclusioni, limiti e sviluppi futuri

Il lavoro svolto ha dimostrato con successo la fattibilità e l'efficacia di una piattaforma integrata che combina l'ingegneria dei dati temporali con l'analisi automatizzata delle anomalie per il monitoraggio chirurgico intraoperatorio.

L'adozione della **Medallion Architecture** ha consentito di strutturare il flusso di raffinamento del dato clinico in stadi chiari e governati, isolando la persistenza dei dati grezzi (**Bronze**), la bonifica e il partizionamento (**Silver**) e l'archiviazione ottimizzata su MongoDB Time Series (**Gold**). I benchmark quantitativi hanno evidenziato vantaggi misurabili: una contrazione dello spazio su disco dell'89.1% e un'accelerazione di 3.7 volte nei tempi di risposta delle query temporali.

Al contempo, l'approccio ibrido all'**anomaly detection** ha confermato l'importanza di affiancare le regole cliniche deterministiche (lo **Shock Index** e l'ipotensione severa) ad algoritmi di **machine learning** non supervisionato (**Isolation Forest** e **LSTM Autoencoder**), garantendo una copertura multilivello delle alterazioni emodinamiche.

In un'ottica di onestà scientifica, è fondamentale riassumere i limiti attuali dell'architettura: la soluzione di **data governance** costituisce un'emulazione artigianale dei principi del **data lakehouse**. L'impossibilità di eseguire inserimenti nelle **Time Series Collection** di MongoDB all'interno di transazioni multi-documento impedisce il conseguimento di un'atomicità fisica nativa tra i dati temporali e il catalogo `registry`, obbligando a ricorrere a strategie applicative di compensazione; l'azzeramento dei valori nulli nel livello **Gold** deriva dalla rimozione delle righe vuote e dall'omissione dei campi privi di valore nei documenti BSON; la perdita di continuità temporale correlata alla cancellazione di campioni nel **Silver** impone cautela analitica; l'asse temporale è attualmente ancorato a una data convenzionale fissa (`2020-01-01`), che in un contesto produttivo reale dovrà essere sostituita dalla marcatura temporale esatta dell'anestesia.

Da un punto di vista normativo e clinico, un sistema di supporto alle decisioni cliniche (**Clinical Decision Support System**, CDSS) destinato alla sala operatoria deve sottostare a stringenti requisiti di sicurezza. In ambito europeo, il Regolamento sui Dispositivi Medici (MDR 2017/745) classifica i software diagnostici in classe IIa o superiore, imponendo la validazione clinica della frequenza dei falsi allarmi per prevenire la cosiddetta fatica da allarme (**alarm fatigue**) nel personale sanitario. Inoltre, la gestione dei dati biometrici ad alta fedeltà richiede la rigorosa conformità al GDPR attraverso tecniche di pseudonimizzazione forte e cifratura dello storage a riposo.

Tra le direttrici di sviluppo futuro più promettenti figurano: l'evoluzione verso un'architettura di **streaming data engineering** mediante l'integrazione di broker di messaggistica distribuiti (quali Apache Kafka o Redpanda), per consentire l'ingestione e la segnalazione delle anomalie in tempo reale con latenze inferiori al secondo direttamente durante l'intervento chirurgico; l'esten-

sione del perimetro analitico a ulteriori parametri biologici, quali la temperatura corporea centrale, la concentrazione di anidride carbonica a fine espirazione (capnografia $EtCO_2$) e le frazioni di gas anestetici alogenati; l'introduzione di moduli di spiegabilità per l'intelligenza artificiale (**Explainable AI**), sfruttando ad esempio i valori di Shapley (SHAP) per fornire all'anestesista una motivazione quantitativa trasparente su quali parametri abbiano maggiormente contribuito a generare l'allarme multivariato.